LLMs for UAV Resource Distribution in Disasters: Enhancing Trustworthiness through Formal Verification

A THESIS SUBMITTED TO AUCKLAND UNIVERSITY OF TECHNOLOGY
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF
MASTER OF COMPUTER AND INFORMATION SCIENCES

Supervisor: Kenneth Johnson

Submmited on November 5, 2025

By
Ronan Irvine Gondipon

School of Engineering, Computer and Mathematical Sciences

Abstract

Unmanned aerial vehicles (UAVs) are essential for the effective distribution of emergency resources during disasters, but conventional approaches face dynamic constraints such as energy constraints, environmental uncertainty, and infrastructure damage. In order to enable optimal UAV task assignment, routing, and scheduling, this study presents an LLM-powered AI agent that converts natural language descriptions of disaster situations into formal constraint satisfaction problems (CSPs). Through formal verification, the system guarantees adherence to safety-critical conditions while minimizing flight time and energy consumption by combining large language models (LLMs) like GPT-5 with the Z3 theorem prover.

Keywords: Large Language Models, Unmanned Aerial Vehicles, Disaster Logistics, Formal Verification, Constraint Satisfaction Problems, Trustworthy AI

Contents

1 Introduction **3**
- 1.1 Context and Motivation . . . 3
- 1.2 Technical Background . . . 3
- 1.3 Problem statement . . . 3

2 Methodology **4**
- 2.1 System Workflow and Components . . . 5

3 Literature Review **9**
- 3.1 Literature Search Strategy . . . 9
- 3.2 Significance . . . 9
- 3.3 Search Query Formation . . . 9
- 3.4 Search Process . . . 10
- 3.5 Inclusion and Exclusion Criteria . . . 10
- 3.6 Summary of the Literature . . . 10
  - 3.6.1 LLM-Driven Drone Coordination in Disaster Scenarios . . . 10
  - 3.6.2 Ensuring Trustworthy and Verified LLM Navigation Strategies . . . 11
  - 3.6.3 Constraints Addressed in LLM-Drone Systems . . . 13

4 Multi-UAV Mission Planning Problem **13**
- 4.1 Case Study . . . 14
- 4.2 Logical Model . . . 15
  - 4.2.1 UAV Mission Planning Constraints . . . 17

5 Z3 Implementation and Problem Resolution **19**
- 5.1 Z3 Constraint Synthesis . . . 19

6 Approach:Programming A LLM-powered AI agent **20**
- 6.1 Parameter Input and Constraint Generation . . . 20
  - 6.1.1 Comparison of Natural Language and Pseudocode Constraints . . . 26
- 6.2 User Prompt . . . 29
- 6.3 Code Generation . . . 29

7 Discussion **32**
- 7.1 Comparison of Natural Language and Pseudocode Constraints . . . 32
  - 7.1.1 Generated Z3 Constraints from Natural Language . . . 33
  - 7.1.2 Generated Z3 Constraints from Pseudocode . . . 37
- 7.2 Generated Code Comparison . . . 40
  - 7.2.1 Generated Code from Using Natural Language Constraints . . . 40
  - 7.2.2 Generated code from Pseudo-code . . . 53
- 7.3 Results Comparison . . . 64
  - 7.3.1 Results from using Natural language constraints . . . 64
  - 7.3.2 Results from using Pseudocode . . . 67
- 7.4 Limitations and Future Work . . . 68

8 Conclusion **69**

1 Introduction

1.1 Context and Motivation

Emergency resources, such as food, medical supplies, and first aid, must be distributed and allocated quickly after disasters like earthquakes, floods, or wildfires to save lives. Traditional ground-based logistics are sometimes impractical due to destroyed infrastructure or possible safety hazards in hard-to-reach or hazardous areas.[GC25] A fleet of Unmanned Aerial Vehicles (UAVs) or drones provides a robust means of quick airborne delivery, allowing access to areas affected by earthquakes or distant flood zones. [SLA+25]

A major challenge is assigning and programming flight paths for UAVs to carry out one or more resource distribution tasks in a way that minimizes flight time and UAV power consumption while efficiently distributing resources. Dynamic constraints such as limited energy, processing power, bandwidth, environmental uncertainties which include strong weathers and obstructions, therefore real-time decision making must all be taken into consideration.[RWRL24] Even if other systems uses an extra constraint validation layer, formal verification techniques and certified fail-safes are still required to step in when LLM outputs diverge from safety-critical constraints. [MKBM25]

1.2 Technical Background

The integration of Large Language Models (LLMs) with drones is advancing rapidly, merging artificial intelligence with robotics to address these challenges in disaster scenarios, such as aid delivery, search and rescue, and communication restoration during events like earthquakes, floods, or wildfires. [JFHS24] LLMs excel at converting high-level commands that are in natural language instructions into executable code and API calls or mission scripts for UAV operations through multi-step logic and reasoning.[ZLZ+24, HCL+25]

These applications face constraints including computational limits, energy, environmental uncertainties, and routing/trajectory planning, where LLMs enable reasoning, adaptive planning, multi-agent coordination, and natural language interfaces. [CEKT25] A key emerging application is the use of LLMs to generate executable code for UAV operations, enabling autonomous or semi-autonomous resource delivery in disaster zones. [Sez25] However even with the executable code, it may not be trustworthy

1.3 Problem statement

In the immediate aftermath of a disaster, the allocation and distribution of emergency resources such as first aid is critical for saving lives. However, resources may often need to be delivered to areas that are difficult reach or hazardous areas, such as flood zones or sites severely damaged by an earthquake. In these situations, unmanned aerial vehicles (UAVs) or drones provide a useful tool for delivering resources to locations deemed unsafe for personnel to access.

Although autonomous AI systems are improving with every iteration, the technology is not currently trustworthy to fully automate task assignment, as it may produce hallucinations, lack numerical precision, or fail to explain its decision-making process.

[SLA+25, HX25] Domain experts therefore require explainable and traceable decision making to ensure reliability in mission-critical scenarios. Therefore, several studies are needed to evaluate their effectiveness with the use of formal verification methods. [CEKT25]

Disaster resourcing in highly dynamic scenarios is challenging. First, UAVs must be assigned resource distribution missions based on their operational state: for example, UAVs low on energy are doomed to fail if assigned mission locations that are too far away. Secondly, UAVs must adapt their navigation to uncertain environmental conditions: weather, smoke and flight-path obstacles need to be handled effectively. Third, the constraints faced throughout the mission must be satisfied before proceeding so as to ensure the success of the mission.

In this research, an LLM-powered AI agent agent is programmed to convert the current disaster scenarios using natural language artifacts capturing key elements of the disaster scenario, which includes the UAV's availability, resource demands, and environmental constraints. The agent then creates formal logical models that depict possible UAV assignments and flight trajectories from these objects. These models are directly verified within the Z3 constraint solver, which formally evaluates their feasibility, optimizes task distribution and guarantee adherence to the constraints presented. Formal verification is central to ensuring that the solver produces the correct feasible solutions with additional measures through tests are required to ensure safety and reliability in the generated code for real-world scenarios.

The remainder of this dissertation is structured as follows: Section 4 formulates the research questions and the literature search strategy. Section 3 details the methodology, literature on LLM-driven drone coordination and trustworthiness/verification. Section 4 presents the approach at for programming the AI Agent with mathematical constraints. Section 5 describes the experiments and results. Section 6 discusses findings, limitations, and future work, concluding with implications for disaster response logistics.

RQ1 How is generative AI used to command and coordinate UAV fleet navigation during a disaster?

RQ2 How are UAV fleet navigation strategies for resource distribution generated from AI shown to be trustworthy?

RQ3 How are formal methods used to verify resource distribution navigation strategies generated by AI

2 Methodology

This study adopts a design science research (DSR) [Wie14] methodology to develop and evaluate a trustworthy system for autonomous UAV resource distribution in disaster scenarios. The methodology integrates formal methods, leveraging a Constraint Satisfaction Problem (CSP) framework and the Z3 theorem prover, with a Large Language Model (LLM, specifically GPT-4o) to translate natural language inputs into structured mission plans ((S)) comprising of routes, schedules payloads along with the constraints that will be done in natural language and pseudocode to test the LLM's translation

accuracy into Z3-readable expressions. The LLM is then used to model the Z3 constraints to ensure trustworthiness through formal verification of well-structured models and in the process, generating artifacts that are auditable. All interactions with the LLM and the resulting models and artifacts are logged. Formal verification is embedded through the Z3 solver. Z3 is a Satisfiability Modulo Theories (SMT) solver developed by Microsoft Research and has been used extensively in software analysis and verification, including predicate abstraction, test case creation, and extended static generation.[dMB08] In this research, Z3 will be used to specifically see if the output plans are not only satisfiable, but also not violate any constraints.

2.1 System Workflow and Components

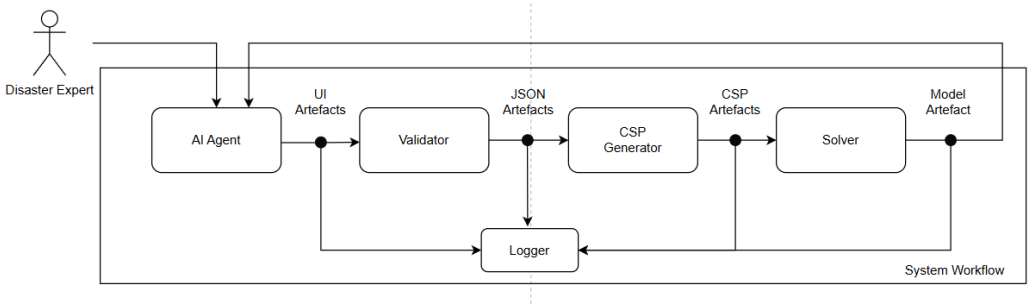


Figure 1: System Workflow

Figure 1 depicts the system workflow, comprising four key components, which continually processes natural language input from a disaster manager, resulting in an audit trace of models and artifacts being generated. These components are the

- AI Agent component, programmed to obtain all relevant information about the disaster scenario, and current state of resourcing, UAV statuses,
- Validation component, which formats the information gathered by the AI Agent about a Disaster Scenario into a JSON object. The JSON object is validated to confirm all data necessary to model the scenario has been obtained
- Constraint Programming Model Generator component is used to
- Solver Component
- Logger Component will store all artifacts including the user interaction artefact, JSON artefact, CSP artefact and model artefact.

Initially, the user begins with natural language input which interacts with the AI agent which processes this input to create a structure problem artifact, which is logged, and when asked to solve will synthesize automatically using structured outputs to create the CSP to be inputed into Z3. The output of Z3 is further processed by the agent to facilitate the user's understanding and can potentially update the problem statement based on the output.

Every workflow stage produces tangible, versioned outputs known as software artifacts, which are then saved for auditing, reusing, debugging, and repeatability. By

recording decisions and allowing re-execution without re-parsing, they act as verifiable intermediaries between machine execution (Z3 models) and human intent (natural language), facilitating formal verification.

1. Natural-language input (User artifacts) The process begins with the disaster manager who describes the disaster scenario in natural language. For example, consider the following input from the user and response from the AI Agent:

```
Hi, I have 5 locations, labeled a to e. c has 15 resources;
The distances between the location are
a to b is 3, b to c is 4 and all other edges are 1.
I have one drone that can carry two resources. It's battery is current at 100%
```

This text constitutes a *user input artifact*, which is time-stamped and logged.

While the AI Agent accepts any text, it must obtain essential information that coincide with the following keywords: `nodes`,`resources`, and `drones` or `UAV`. This is so that the agent can properly extract all the information relevant to the resourcing problem. This workflow contains both static and dynamic aspects:

- **Static Aspects**: The LLM (prompted via `SAMPLE_PROMPT`) follows a fixed schema for output (Keys Parameters: "nodes", "edges", "resource locations", "resource demands", "constraints" as Z3-logical strings). Defaults (Example: time = distance, max_trips=6) are hard coded in the prompt.
- **Dynamic Aspects**: The parser adapts to any text input, auto-corrects typos (example:'$a - c >$' to '$a \rightarrow c$'), flags ambiguities (example: missing edges), and prompts users for clarification. Pydantic models (example, `ProblemParams`) enforce type safety and custom validators (example, ensure source node exists, demands are positive).

2. LLM parser The LLM validates the natural-language description and outputs a structured parameter set and an initial set of formal constraints: nodes, depot/source, edge distances and times, resource availability and demands, drone specifications. To ensure the system receives a complete input, each optional parameter (maximum trips, time gap between flights and service time) contains default values. Should there be any missing important parameters(such as nodes, edges, resource location, quantity, or drone details) or vague constraints, the system will prompt the user for more information and clarification. To evaluate the LLM's translation accuracy, two variants of the SAMPLE_PROMPT were tested: one using natural language descriptions for constraints and another using pseudocode mathematical expressions. This component balances static and dynamic elements:

- **Static behavior:** The parser follows a fixed, prompt-defined output schema (labels, formatting), so downstream components receive a predictable representation.
- **Dynamic behavior:** The parser adapts to natural language, flags ambiguous or missing information such as missing edges, or ambiguous node labels, fixes simple typos (example: I have 5 nodes from a to e',$a - c>$' to '$a - >c$'), and prompts the

user for clarifications when necessary. The result is a verified machine-readable parameter dictionary as the artifact. This will allow the fuzzy input to turn into machine readable parameters, reducing the effort of the user on specifying exact details.

3. Structured Parameter Generation The output from the LLM is then converted into a `JSON` object, where the nodes, source location, available resources, areas with resources needed, edges (routes with distances and time), and drone specifications are all identified into.

The `Pydantic` library is used to verify and transform the structured `JSON` output into internal Python data structures. These structures offer robust type validation and consistency checks and are described by models like `Edge`, `Drone`, and `ProblemParams`. For instance, the system makes sure that all edges have defined lengths and times, that drones have proper capacity and energy values, and that the source node is present among all declared nodes.

4. Constraint Programming Model (LLM Modeled Problem) The validated parameters from the JSON artifact are then fed into an LLM-guided constraint programming model, formulated as a constraint satisfaction problem (CSP) formulated as $CP = (V, D, C)$ with variables $V = (\Pi, Y, R)$ (routes via $X_{i,j}^{k,t}$, schedules via $y_i^{k,t}$ or arrival/delivery times, resources via $r_i^{k,t}$ or deliveries), finite domains D (example: trips bounded by max_trips, energies), and constraints C parsed from the 'constraints' list in params (Z3-logical strings from LLM parser).

Guided by `CODE_GEN_PROMPT`, the LLM generates a tailored Z3-compatible Python function `run_z3_solver(params)` as a CSP artifact, which encodes the boolean variables for edge/trip selection, integer variables for arrival/delivery times, resource variables, and total energy consumption. Constraints related to timing and logistics logic to prevent clashing are also encoded without the user having to consider them. To assess LLM translation fidelity, two variants of the SAMPLE_PROMPT were employed: one using natural language descriptions (example: "Each drone trip must start at or after time zero") and another using pseudocode mathematical expressions (example: start_time$[k, t] \geq 0 forall k, t$).

This modeling exhibits:

- **Static Aspects**: Base CSP structure, variable templates, constraint operators (ForAll, Implies, Sum), and optimization goals (min energy then trips) are fixed in the prompt, ensuring adherence to the CSP model($V = \{\Pi, Y, R\}, D, C$) and fixed schema, presenting a detailed solution table with routes, start/end times, delivery times, energy usage, and resources delivered.

- **Dynamic Aspects**: The generated code scales with parameters containing loops over parsed variables (nodes/drones/trips), and substitutes optional parameters(maximum trips, time gap between flights and service times) with defaults if unspecified. Constraints from the JSON file are translated into Z3 expressions to be put into the Z3 solver. Syntax and bracket errors trigger a repair loop (up to 5 retries), where errors are fed back to the LLM for code regeneration. The code validation

uses `ast.parse()` for syntax checking and a token-based function for bracket matching before execution.

5. Z3 Solver Execution (Model Artifact) The Z3 solver's function (`check()`) processes the CSP, yielding a model artifact if satisfiable or unsatisfiable(unsat core) for diagnostics. Iterative minimization refines bounds until optimal (lowest energy/trips), with timeouts logged. After retrieving a solution, a nested function `find_path` will be used to output paths provided by the solver.

Static/dynamic integration:

- **Static Aspects**: Core flow includes the prompt defined solver, built-ins modules to use and table based outputs are fixed in the prompt ensuring consistent search and regex-validated format (such as prompt-defined Optimize/Solver, safe built-ins for exec restriction, table-based output with starting node/ending node/deliveries/ energy cost) and bounds (such as maximum trips max_trips) are fixed in prompt, ensuring consistent search and regex-validated format

- **Dynamic Aspects**: Variables and constraints adapt to input parameters such as available routes between nodes defining possible drone paths and required resources at each node enforces an exact amounts required. If no feasible plan is found after retrying, the AI will suggest changes such as increasing drone energy or reducing the distance between nodes. The code generation will attempt to repair crashes or syntax errors on-the-fly up to 5 times with all of them being logged. After a successful solve, another function `find_path` rebuilds the drone routes by following the solver's successful selected paths.

6. Mission Plan Output (Plan Artifact) The extracted model formats into a per-drone summary containing trip tables (paths, timings, resources held/delivered, energy used/remaining, existing demands), which is a human-readable artifact of the mission plan. In order to finish the loop, total metrics (energy, trips, and delivered) are saved as text for auditing.

- **Static Aspects**: Fixed output template ensures consistency and verifiability against Z3 model.

- **Dynamic Aspects**: Plans adapt to solution scale containing multi-trip sequences, and cumulative energies, with real-time solve times and failure suggestions via second LLM call.

Rationale for Using Two Prompts: To address the UAV routing problem, two distinct prompts were employed : one for input parsing and one for code generation. The input parsing prompt extracts and structures the disaster manager's input into a JSON artifact, including problem parameters and Z3-readable constraints, ensuring standardized data representation. The code generation prompt produces the Python function `run_z3_solver`, which models the UAV routing problem with decision variables and constraints and implements Z3 solver logic to compute optimal solutions. The use of two prompts allows for modularity by separating input processing from solver implementation, enabling independent development, debugging, and scalability, while

also reducing token usage in the LLM processing by dividing complex tasks, minimizing computational costs.

3 Literature Review

3.1 Literature Search Strategy

A comprehensive search strategy was developed, utilizing Google Scholar, IEEE and ScienceDirect for academic databases, and hand-searching several documents in search for the appropriate literature, articles, and conference papers.

3.2 Significance

By evaluating several publications that cover LLM-driven logistics and optimization, this study makes a substantial contribution to the application of large language models (LLMs) in disaster resource distribution using drones, demonstrating how formal verification ensures reliable strategies for real-time deployment, optimized flight paths, and dynamic resource allocation.

The study demonstrates how LLMs integrated with formal verification can coordinate real-time drone deployment, optimize flight paths, and allocate resources dynamically in disaster scenarios while guaranteeing reliable distribution strategies. Optimization is done through the Z3 solver's minimize function which is able to total travel distance and energy consumption while satisfying constraints on time, capacity, and delivery feasibility.

These contributions are essential for addressing the increasing frequency and severity of natural disasters, ensuring that drones can deliver aid quickly and equitably to affected communities, while also enhancing disaster management strategies that reduce financial losses and save lives.

3.3 Search Query Formation

To capture exact phrases and guarantee that the search results concentrate exclusively on those phrases, keywords included double quotation marks in their inquiries. Results may be less accurate if the individual words are not included in quote marks since they might appear anywhere in the results. This method aids in search refinement and yields more appropriate outcomes. Several combinations of keywords were used to find the most appropriate results includes:

1. LLM
2. UAV
3. Drone
4. disaster resource AND (management OR distribution OR delivery)
5. Formal Verification

Queries using the keywords above resulted in too many results some with some results being out of the scope of the research. More specific set of keywords were further used to obtain more relevant results

"LLM" AND ("UAV" OR "drone") AND Formal Verification AND (disaster resource AND (manager

3.4 Search Process

Exploration: An overview of the types and number of results retrieved was obtained using the identified keywords.
Adding and refining keywords: To focus on the words that were particularly relevant to the research topic, keywords were added and refined based on the initial search results.
Create query: To do searches in the Google Scholar and ScienceDirect, IEEE database, a Boolean expression was created using the selected and refined keywords.
Database search: Only publications published between 2020 and 2025 were found in Google Scholar using the query.
Gathering reference materials: A quick evaluation of the articles that were relevant to the research was conducted by scanning and skimming through the abstract of these articles.

3.5 Inclusion and Exclusion Criteria

Inclusion Criteria	**Exclusion Criteria**
English-language publications	Non-English publications
Publications from 2020 to 2025	Publications before 2020
Peer-reviewed journal articles and conference papers	Books, theses, government reports, and opinion articles

Table 1: Inclusion and Exclusion Criteria

Specific inclusion and exclusion criteria were established to guarantee only quality and pertinent material was reviewed. This strategy helped to avoid pointless research and focusing on the most beneficial ones. The specific requirements are listed in Table 1.

3.6 Summary of the Literature

We summarized the literature according to topics: LLM-Driven Drone Coordination in Disaster Scenarios, Trustworthiness and Formal verification and Constraints Addressed in LLM-Drone Systems.

3.6.1 LLM-Driven Drone Coordination in Disaster Scenarios

For navigation strategies to be reliable, UAVs must dynamically adapt to rapidly changing conditions, including unpredictable terrain and electromagnetic interference. UAVs empowered with LLMs can not only optimize flight paths in real-time but also anticipate and avoid potential hazards based on learned patterns, enhancing mission safety and effectiveness[JFHS24].

Recent research emphasizes language-driven command generation, where natural language processing and retrieval-augmented generation (RAG) enable LLMs to dynamically plan and adapt missions in complex disaster scenarios. [Sez25] integrates RAG with LLMs to process telemetry, mission logs, and environmental data, generating adaptive commands like "Reroute to avoid storm" for hazards or "Evacuate northwest for turbulence" in high-wind or GPS-denied scenarios, supporting safe aid distribution paths in flood or wildfire contexts. Similarly, [ZLZ+24] uses GPT-3.5 in MultiBotGPT to translate commands such as "look for where location number 6 is?" or "UGV arrives under UAV" into executable tasks via structured prompts from a Robotics Competency Library, facilitating UAV-UGV collaboration with Theta*-APF for obstacle avoidance and path smoothness. [JPDB25] extends this by combining RAG with multi-modal LLMs and FrameNet for semantic annotation, generating continuous, unprompted natural language insights from UAV imagery (example: identifying victims or hazards) in Brazilian flood search-and-rescue (SAR), prioritizing operator-assisted missions.

In contrast,the integration with other models with swarm-based planning and the incorporation of sensory data enhances situational awareness. In [JFHS24] a transformer-based LLMs such as GPT, and BERT were employed for route optimization and swarm intelligence, processing weather and terrain data to reassign tasks in disrupted infrastructures like earthquakes or floods, ensuring efficient operational resource allocation. While [MKBM25] uses a dialog-driven framework, integrating multimodal inputs such as LiDAR, and thermal sensors with reinforcement learning (RL) and linear programming to plan resource(foam, water drop) delivery, addressing physical constraints such as thermal limits, energy thresholds, safety boundaries avoiding structural hazards and high-pressure zones, and operational constraints related to terrain topology and mechanical failures.

Simulation-driven and optimization approaches are also possible, especially with the incorporation of sensory data and coordination constraints. [LMSK24] uses LLMs as cognitive agents in agent-based simulations for wildfire System of Systems (SoS), dynamically adjusting UAV roles such as surveillance or support via prompt-driven operational concepts (ConOps) that include weather checks, evacuation procedures, and communication protocols. It models resource allocation (example: water drops, evacuation) in grid-based terrains, leveraging sensory data for situational awareness, but struggling to assess critical factors like fire magnitude, limiting response scaling [LMSK24]. [GC25] hybridizes LLMs with genetic algorithms in the Loegised framework, embedding routing constraints such as depot limits, drone endurance, and takeoff/landing rules into prompts to generate JSON-formatted mission plans while assessing route difficulty in mining environments using multimodal data (example:, temperature, humidity, wind speed), enabling predictive trajectory adjustments akin to disaster exploration. [**?**] complements these by integrating algorithms like Adaptive Belief Tree (ABT) for POMDP-based navigation and SLAM for localization, enhancing adaptability for resource distribution in mission-critical networks like earthquake or flood zones.

3.6.2 Ensuring Trustworthy and Verified LLM Navigation Strategies

The reliability of LLM-generated navigation strategies in autonomous UAV operations is assessed through the quality of command generation across challenging scenarios, including high winds, GPS signal loss, object detection, battery depletion, and adherence

to operational constraints. Evaluation metrics commonly include accuracy, robustness, and mission success rates, often benchmarked against traditional methods, while highlighting the challenges posed by dynamic environments.

For example, [Sez25] evaluates the relevance of the command using cosine similarity (0.87 average) and BLEU scores (0.82 average), showing that LLM-generated commands are both accurate and contextually appropriate for the execution of a specific mission. When combined with retrieval-augmented generation (RAG), the model adapts dynamically to real-time situations and outperforms classical classifiers such as Decision Trees, Neural Networks, and Random Forests by 10–20% in dynamic scenarios. Similarly, [ZLZ+24] demonstrates that integrating GPT-3.5 into MultiBotGPT enables natural language commands (example:"Drones look for where location number 6 is?") to be translated into executable multi-robot tasks, coordinating UAVs and UGVs with a 94.4% success rate—surpassing BERT-based methods by 55%—while reducing operator strain and maintaining task efficiency.

Other studies emphasize layered validation and human-in-the-loop oversight to strengthen trust. [MKBM25] employs a lightweight linear programming (LP) module as a constraint validator, checking RL-generated action sequences against live operational constraints. Violations to the constraints will trigger a fallback mechanism where the LLM revises the plan, effectively acting as a semantic co-pilot. Building on this, [JPDB25] highlights the critical role of human operators in high-stakes missions, such as search and rescue, where situational awareness, multitasking, and decision-making under pressure are essential. Human oversight provides an additional safeguard, enabling either the validation or rejection of the outputs before execution.

Formal verifications are paramount to ensuring LLM outputs for safety-critical drone operations, mitigating physical collisions and unsatisfied constraints or unmet delivery constraints. Based on this need, several studies combine quantitative evaluation with formal validation techniques. An example from [LMSK24] observes qualitative improvements in decision richness and resource utilization in wildfire simulations when compared to decision trees, although also notes limitations such as bias and reliance on sensor integration. In contrast, [GC25] provides a statistical parallel, using Wilcoxon tests for 20% delivery time reductions, but limits to logistics benchmarks, unlike the disaster focus of others. The use of a Constraint Satisfaction Index (CSI) further ensures the feasibility of the solutions generated, evaluating all problem constraints through indicator functions so that only solutions fully satisfying every delivery and operational constraint are accepted for further testing [GC25] .

In the scenario of controlling the spread of fires, [MKBM25] encodes safety constraints, such as thermal limits in fire response, as linear inequalities ($Ax \leq b$) where A encodes operational boundaries (example: thermal propagation, energy use) and b specifies limits (example: maximum temperature thresholds) and where x represents the sequence of planned actions over the decision horizon. A lightweight LP module validates these constraints in real time within a Constrained Markov Decision Process (CMDP), triggering LLM-assisted re-planning when violations occur. Likewise, [HX25] proposes a dual-layer validation strategy in which an additional LLM checks code line by line for constraint compliance, further enhancing reliability and enabling automatic

intervention near safety boundaries.

Although not all studies explicitly focus on disaster-driven resource distribution, the strategies and verification mechanisms described here strategies can be applied to other studies. Ensuring both trustworthiness and verifiability of LLM navigation strategies is essential for deploying UAVs in safety-critical contexts, where reliable cooperation under resource and time constraints can determine mission success.

3.6.3 Constraints Addressed in LLM-Drone Systems

Beyond routing, logistical, timing, and environmental constraints critically shape UAV operations for delivering resources such as medical supplies, water, and food to priority disaster zones, ensuring rapid, collision-free allocation. Logistically, UAVs face finite battery life, payload capacity, and communication radius, necessitating efficient task division and load balancing to prevent overload or disconnection, as seen in multi-robot exploration where drones maintain ad-hoc networks and offload computations to edge/cloud servers [**?**, HCL+25].

Regional accessibility restrictions, such as hazardous terrains or regulatory hurdles, require adaptive role assignments such as surveillance to relay to ensure coverage without overlap [RWRL24, GC25]

Timing constraints are paramount to avoid physical collisions, requiring precise synchronization of UAV takeoffs, landings, and service schedules[Sez25]. For instance, takeoff and landing restrictions prevent drones from occupying the same location simultaneously, while a minimum time separation between takeoffs ensures collision-free departures. [GC25]. Real-time inference demands low-latency communication (1–10 ms control cycles) to align multi-modal inputs, as a 300 ms misalignment between vision/language data can increase task errors by 28% [HX25, **?**].

Rapid task restarts in dynamic networks and optimized trajectories further minimize delays, enabling UAVs to navigate disrupted landscapes while respecting energy budgets and service windows to deliver aid efficiently [HCL+25, **?**].

[GC25] handles harsh mining environments such as GPS-denied tunnels, or weather through multi-modal data integration, evaluating conditions via sensor fusion (temperature, humidity, wind) for predictive modeling and adaptive navigation; LLMs assess route difficulty by analyzing weather, terrain, and performance metrics, enabling predictive adjustments to trajectories and coordination with planning algorithms for collision avoidance.

[GC25] further emphasizes environmental constraints, noting that LLMs may be used to assess the degree of difficulty of operating circumstances. Through analyzing historical and current meteorological data as well as vehicle performance indicators. The LLM can evaluate the difficulty of the current situation and predict future difficulties by incorporating sensors on the UAVs to collect data on temperature, humidity, wind speed, and other environmental variables.

4 Multi-UAV Mission Planning Problem

The problem considered in this study is a multi-UAV mission planning and routing optimization task for payload delivery across a network of nodes. The goal is to generate

feasible schedules and efficient flight routes while satisfying operational constraints such as safety, timing, and service requirements.

4.1 Case Study

To illustrate the problem, a disaster relief scenario is considered with a set of interconnected locations (nodes). One node acts as the depot (base station) which contains all the resources and the remaining nodes as delivery points have specific resource demands that must be met within a given mission time UAVs are stationed at the depot, each with limited carrying capacity and battery energy. The UAVs must be able to deliver the resources The effects of payload weight and environmental conditions are abstracted into the traversal cost of each edge.

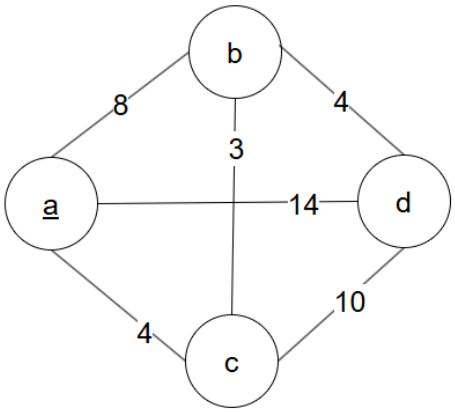


Figure 2: Disaster-Affected Region Case Study

Figure 2 shows a disaster area consists of four nodes a, b, c, and d interconnected by a set of directed edges $E = \{a \to b : (8,8),\ a \to c : (4,4),\ \ldots\}$ with associated travel distances and times. The underlined node a represents the source depot where the resources $(a : 5)$ are initially stored, and where the UAV's are located while nodes b, c and d are affected sites requiring resources deliveries $(b : 2,\ d : 3)$.
The whole edge set E is given by:

$$E = \{\, a \to b : (8,8), \quad a \to c : (4,4), \quad b \to d : (3,3), \quad c \to d : (10,10), \quad a \to d : (14,14),$$
$$b \to a : (8,8), \quad c \to a : (4,4), \quad d \to b : (3,3), \quad d \to c : (10,10), \quad d \to a : (14,14)\,\}.$$

The two numerical values in each tuple denote the distance and corresponding travel time between nodes. Although the network is modeled as directed to enforce path validity and logical route constraints, the travel costs between nodes are symmetric; that is, the distance and time from node a to b for example are equal to those from b to a. For this case study, the scenario is scaled such that 1 unit of distance corresponds to 1 kilometer and 1 unit of time corresponds to 1 minute, implying that the UAV's effective speed will be 60 km/h. For simplicity, the effects of payload weight on flight performance are neglected, and UAV battery/energy cost is based on total distance traveled, which ensures that each UAV can complete its assigned route within its available energy capacity.

This simplified topology allows the solver to explore all feasible drone assignments and delivery routes while minimizing overall travel cost and energy consumption.

4.2 Logical Model

To solve the problem, we devise a logical model featuring a state space of variables describing the positions and state of the disaster affected area, following by constraints which codify the set of legal movements of the UAVs so they satisfy mission-critical requirements. Codify mission critical requirements into formula. In this disaster relief mission, a proactive mission plan S is sought, specifying routes, flight schedules, and resource deliveries to ensure that the required items are delivered within the time frame while respecting operational and environmental constraints.

The system is modeled as a directed graph to represent the direction of UAV routes and enforce routing constraints even though the distances and travel time between nodes are symmetric.

$$G = (N, E), \tag{1}$$

where $N = \{1, \ldots, n\}$ is the set of interconnected nodes and $E = \{\{i, j\} \mid i, j \in N, i \neq j\}$ is the set of directed edges. Node 1 denotes the depot (base station), while the remaining nodes correspond to delivery locations all with differing cost from one another.

The variables associated with the network are defined as follows:

- z_i — demand at node $i \in N$, with $z_1 = 0$ (depot),
- $d_{i,j}$ — distance between node i and node j,
- $t_{i,j}$ — travel time between node i and node j,
- w — service or waiting time required at each delivery node,
- ts — minimum time separation between UAV takeoffs from the base.

The decision variables are as follows:

- $s^{k,t}$ — takeoff time of UAV k on trip t, for $k \in K$, $t \in \{1, \ldots, \text{max_trips}\}$,
- $y_i^{k,t}$ — arrival (or service start) time of UAV k at node i on trip t,
- $x_{i,j}^{k,t}$ defined as

$$x_{i,j}^{k,t} = \begin{cases} 1 & \text{if UAV } k \text{ travels from node } i \text{ to node } j \text{ on trip } t, \\ 0 & \text{otherwise,} \end{cases}$$

- $r_i^{k,t}$ — resources delivered by UAV k to node i on trip t,
- bat^k — total energy consumed by UAV k across all trips,

- π_k^t — route of UAV k on trip t:

$$\pi_k^t = (v_1, \ldots, v_i, v_{i+1}, \ldots, v_\mu), \quad v_i \in N,\ x_{v_i,v_{i+1}}^{k,t} = 1$$

The UAV mission planning problem is formulated as a Constraint Satisfaction Problem (CSP), defined as

$$CP = (V, D, C), \tag{2}$$

where

- **Variables** $V = (\Pi, Y, R)$ — a set of decision variables:
  - Π: possible UAV routes,
  - Y: delivery schedules,
  - R: amount of resources delivered by each UAV to each recipient node.
- D — finite domains of the variables, representing feasible values (example: whether a given assignment is possible or not).
- C — a set of constraints that specify the relationships between UAV routes, UAV schedules, and transported resources (formulas (2-11) seen in 4.2.1.

The CP models the problem of finding a proactive mission plan S, which is a specific solution assigning values to the variables in V to deliver essential items during disaster relief missions. The mission plan S comprises:

- **Routes** (π_k^t): Sequences of nodes (example, $a \to b \to d \to a$) visited by each UAV k on trip t, ensuring all demands z_i are met via resource deliveries.
- **Schedules** $(s^{k,t}, y_i^{k,t})$: Timestamps for UAV takeoffs and deliveries, adhering to time separation (ts), service times (w), and maximum mission time $(t_{\max})$.
- **Payloads** $(r_i^{k,t})$: Resources delivered to each node, satisfying demands $(\sum_{k,t} r_i^{k,t} = z_i)$ within UAV capacity limits.

The plan S ensures timely delivery before battery depletion, respecting energy constraints ($bat^k \leq \text{energy}_k$), and is robust to environmental factors, such as weather, which are abstracted into edge costs $(d_{i,j}, t_{i,j})$ or time bounds in this model.

For example, the mission plan S for a scenario with nodes $\{a, b, c, d\}$, source node a, resources available $a : 5$, and resource demands $b : 2$, $d : 3$. The edges in the network have distances $a \to b : 8$, $a \to c : 4$, $b \to d : 3$, $c \to d : 10$, $b \to a : 8$, $c \to a : 4$, $d \to b : 3$, $d \to c : 10$, $a \to d : 10$, $d \to a : 10$, and the UAVs have capacities and energy limits $d1$ (capacity 2, energy 50) and $d2$ (capacity 1, energy 50). The plan assigns UAV $d1$ two trips, $\pi_1^1 = (a, b, d, a)$ and $\pi_1^2 = (a, d, a)$, and UAV $d2$ one trip, $\pi_2^1 = (a, d, a)$.The plan satisfies all node demands ($z_b = 2$, $z_d = 3$) and adheres to UAV capacity, energy, and scheduling constraints, as illustrated below:

$$\begin{cases} \text{Drone 1, Trip 1:} & \pi_1^1 = (a, b, d, a),\ r_b^{1,1} = 1,\ r_d^{1,1} = 1 \\ \text{Drone 1, Trip 2:} & \pi_1^2 = (a, d, a),\ r_d^{1,2} = 1 \\ \text{Drone 2, Trip 1:} & \pi_2^1 = (a, d, a),\ r_d^{2,1} = 1 \end{cases} \tag{3}$$

This plan enables UAV $d1$ to deliver 1 resource to node b and 1 to node d in its first trip, 1 resource to node d in its second trip, and UAV $d2$ to deliver 1 resource to node d in its first trip, satisfying the demands ($z_b = 2$, $z_d = 3$) while respecting energy and capacity constraints. The total energy consumption for $d1$ is $d_{a,b} + d_{b,d} + d_{d,a} = 8 + 3 + 10 = 21$ for trip 1 and $d_{a,d} + d_{d,a} = 10 + 10 = 20$ for trip 2 (total 41), and for $d2$ is $d_{a,d} + d_{d,a} = 10 + 10 = 20$, both within the energy limit of 50.

4.2.1 UAV Mission Planning Constraints

The model is subject to the following constraints. The routing constraints used in this experiment are based on the routing constraints used in [RGDB+21]. The original constraints were meant for single trip multi-UAV deliveries, but have been adapted to for multi-trip deliveries.

$$s^k \geq 0, \quad k = 1, \ldots, K \tag{4}$$

Constraint (2): $s^k \geq 0$ for all $k = 1, \ldots, K$. Each UAV's mission start time must be non-negative. Here, s is the start time and k indexes the drone. This ensures that the take-off or mission start time of each UAV/drone k is non-negative.

$$y_i^k \geq 0, \quad i = 1, \ldots, n; \quad k = 1, \ldots, K \tag{3}$$

Constraint (3): $y_i^k \geq 0$. For all UAV arrival or service times at delivery nodes must be equal to or greater than zero enforcing temporal consistency for the schedule.

$$(|s^k - s^q \geq ts|); k, q = 1, \ldots, K; \quad k \neq q \tag{4}$$

Constraint (4): $s^k \neq s^q$ for all $k \neq q$. UAVs cannot take off from the base at the same time, ensuring no two UAVs start simultaneously.

$$\sum_{j=1}^{n} x_{1,j}^k = 1, \quad k = 1, \ldots, K \tag{5}$$

Constraint (5): $\sum_{j=1}^{n} x_{1,j}^k$ =1 for all k=1,...,K Each UAV's route must start from the base node 1, with exactly one outgoing edge selected. This guarantees that every UAV starts from a common depot.

$$\left(y_i^k \neq 0 \wedge y_i^q \neq 0\right) \implies (|y_i^k - y_i^q| \geq w); \quad i = 1, \ldots, n; \quad k, q = 1, \ldots, K; \quad k \neq q \tag{6}$$

Constraint (6): For each recipient node i, it cannot be served simultaneously by more than one UAV. If multiple UAVs serve the same node, the schedule times must not overlap; only one can deliver at a time.

$$(x_{1,j}^k = 1) \implies (y_j^k = s^k + t_{1,j}); \quad j = 1, \ldots, n; \quad k = 1, \ldots, K \tag{7}$$

Constraint (7): If a UAV k travels directly from node 1 to node j (i.e., $x_{1,j}^k = 1$), then the delivery/arrival time $y_j^k = s_k + t_{1,j}$. Ensures delivery/arrival time at node j accounts for takeoff time plus flight time from the base.

$$(x_{i,j}^k = 1) \implies (y_j^k = y_i^k + t_{i,j} + w); j = 1, \ldots, n; \quad i = 2, \ldots, n; \quad k = 1, \ldots, K \tag{8}$$

Constraint (8): If UAV k travels from node i to node j (with $i, j \neq 1$ and $x_{i,j}^k = 1$), then $y_j^k = y_i^k + t_{i,j} + w$. This ensures the delivery schedule respects flight time $t_{i,j}$ plus service time w between nodes i and j, maintaining temporal fidelity along routes.

$$\sum_{j=1}^{n} x_{i,j}^k = \sum_{j=1}^{n} x_{j,i}^k; \quad i = 1, \ldots, n; \quad k = 1, \ldots, K \tag{9}$$

Constraint (9): For all nodes i and UAVs k, $\sum_{j=1}^{n} x_{i,j}^k = \sum_{j=1}^{n} x_{j,i}^k$. The number of UAVs entering node i is equal to the number leaving node i for each UAV k, ensuring routes are continuous and represent closed paths. Continuity and closed-loop routing are guaranteed by this constraint.

$$y_i^k \leq t_{\max} \cdot \sum_{j=1}^{n} x_{i,j}^k, \quad i = 1, \ldots, n; \quad k = 1, \ldots, K \tag{10}$$

Constraint (10): The delivery time at node i is bounded by a maximum allowed time t_{max} if node i is served by UAV k. Ensures deliveries are completed within the maximum allowed time.

$$x_{i,i}^k = 0, \quad i = 1, \ldots, n; \quad k = 1, \ldots, K \tag{11}$$

Constraint (11): $x_{i,i}^k = 0$ for all nodes i and UAVs k Prevents UAVs from traveling from a node back to the same node (no loops of length zero). This prevents trivial loops from a node back to itself, thereby avoiding redundant or meaningless route segments.

These constraints collectively ensure feasible UAV mission planning, covering safe takeoff, non-overlapping service schedules, route continuity, and operational deadlines. The formulation closely follows related approaches in UAV routing optimization, such as [RGDB+21].

5 Z3 Implementation and Problem Resolution

To automate the process of route finding for multi-UAVs, we translate our logical model into the Z3 specification language. The CSP is solved using Z3 which is well-suited for this problem due to its ability to handle mixed integer constraints, Boolean logic, and iterative constraint satisfaction efficiently [dMB08]. Z3 takes as input logical constraints $K_1, \ldots, K_p$ and works out an assignment $\alpha : X \to \mathbb{Z}$ of the variables X such that the conjunction $K \equiv \bigwedge_{i=1}^{p} K_i$ evaluate to true. In symbols, $[K](\alpha) = true$, which we write as $K \models \alpha$.

In conclusion, constraints (2–11) provide the core operational and logical foundation for UAV scheduling and routing, guaranteeing delivery' validity, continuity, and exclusivity. These are combined with additional energy, resource, and timing constraints to produce a comprehensive model that balances operational safety, feasibility, and efficiency in UAV-based delivery missions.

Listing 4.2.1 presents a fragment of the UAV resource delivery model, encoded in Z3 syntax. The model defines integer and Boolean variables to represent each UAV's start time (s^k), arrival time (y_i^k), routing path ($x_{i,j}^k$), trip status (`trip_active`), resources delivered, and total energy consumption. Constraints (2)–(11) implement the core routing and timing logic of the model. Specifically, Lines 7–17 enforce non-negative start and arrival times, prevent UAVs from departing simultaneously, ensure each UAV starts at the base, maintain temporal consistency for delivery times, and prohibit self-loops. The constraints also preserve route continuity by equating incoming and outgoing edges, and bound all delivery activities within the maximum allowable mission time.

Listing 1 presents a small portion of the UAV resource deliver model for the Z3 model.

Global Objective: The overall optimization objective minimizes total mission cost, which may combine flight time, energy consumption, or total delivery delay across all UAVs and nodes. The solver aims to produce an optimal schedule and routing configuration that minimizes this aggregate cost function while satisfying all operational constraints.

5.1 Z3 Constraint Synthesis

Let $1 \leq k \leq N$, where N is the number of UAVs, and $1 \leq t \leq M$, where M is the maximum number of trips a UAV is allowed. We define the Constraint The resolution process involves the following steps:

1.

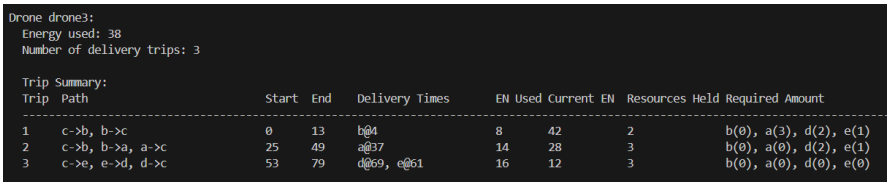
4. Solution From the solution, the mission plan S is constructed by extracting:

- **Routes:** edges where $x_{i,j}^{k,t} = 1$,
- **Schedules:** UAV takeoff and arrival times from $s^{k,t}$ and $y_i^{k,t}$,
- **Deliveries:** resource quantities $r_i^{k,t}$ ensuring coverage of all demand nodes.

This approach ensures that a feasible mission plan is found, if one exists, and iteratively improves it to reduce energy and trips. However, due to the iterative nature, it may

not always find the globally optimal solution with minimal energy and trips.

5. Output: The solution is then formatted as a report with per-UAV trip details (routes, times, deliveries, energy) and a summary table. If unsatisfiable (`unsat`), Z3 provides a core to diagnose issues (example: insufficient energy or invalid routes).



```
Drone drone3:
  Energy used: 38
  Number of delivery trips: 3

  Trip Summary:
  Trip  Path                    Start  End   Delivery Times     EN Used Current EN  Resources Held Required Amount
  ------------------------------------------------------------------------------------------------------------------
  1     c->b, b->c              0      13    b@4                8       42          2              b(0), a(3), d(2), e(1)
  2     c->b, b->a, a->c        25     49    a@37               14      28          3              b(0), a(0), d(2), e(1)
  3     c->e, e->d, d->c        53     79    d@69, e@61         16      12          3              b(0), a(0), d(0), e(0)
```

Figure 3: Satisfiable Output

6 Approach:Programming A LLM-powered AI agent

The proposed system operates in two stages: 1. Parameter extraction from unstructured user input and constraint specification. 2. Automated generation of a complete solvable Z3 model in Python.

By combining natural-language understanding, structured JSON inter-mediation, and code generation, the agent produces verifiable, energy-efficient, and time-feasible UAV schedules all from a single conversational prompt. The following subsections detail each of this workflow's components.

6.1 Parameter Input and Constraint Generation

Prompt Design: The system prompt is designed to guide the LLM (GPT-4o) in parsing natural-language descriptions of disaster scenarios for UAV mission planning. It begins with a clear problem statement, outlining the task of planning drone deliveries from a source node (example: 'c') to recipient nodes (example: 'a', 'b', 'd') to meet resource demands while satisfying constraints like timing, energy, and capacity. The prompt specifies the expected variables such as nodes, source node, resources, edges, drones, and optional constraints like maximum trips or service time and the objective of generating feasible UAV schedules.

System Prompt

```
1 Your role is to assist a disaster manager who must distribute lifesaving resources (food and
      water) within a disaster-affected region using a fleet of UAVs (which we also call drones).
2 Drones have capacities, energy limits, and must follow routes (edges exp: a->b:2 from node 'a' to
       'b') with distances/times, while respecting constraints like no zero-delivery trips, energy
       consumption (equals edge distance), and returning to the depot after each trip.
3
4 You will assist by requesting the following information in 1, 2, 3  and 4 below from the disater
      manager, and generate a JSON formatted file as an artefect when all information is confirmed
      .
5
6 1. a. Information about the region. You may assume the region is represented by a weighted, bi-
      directional GRAPH G, (weighted by physical distance in meters).
7 b. Ask followup details from the disaster manager using basic graph terminology such as nodes,
      edges, weights.
```

```
2. a. The number of drones (k) currently available,
   b. Their source location (source must be the same for all drones)
   c. Each drone's battery power, and capacity for carrying resources
3. Amounts and locations of food and water resource demands throughout the region. All resources must be at the source node, and cannot reside on edges.
4. Additional information such as
    a. Maximum time limit t_max,
    b. Maximum Number of trips a drone can make max_trips. One trip is considered when the drone (k) from the source to other locations back to the source location,
    c. Landing, takeoff, or service time required at delivery nodes (w).
    d. Minimum time gap between drone start times for the same trip (t_s)

Ensure to extract the below information from the user:
- Nodes: List of location names (exp: ["a", "b", "c"]).
- Source Node: Node with resources (exp: "c").
- Resources Available: Dict of resources at source (exp: {"c": 15}).
- Resources Required: Dict of resources needed at nodes (exp: {"b": 4, "a": 5}).
- Edges: Routes with distances and times (exp: [{"from": "a", "to": "b", "distance": 3, "time": 3} or {"a"->"c":5" means a to c takes 5 distance and 5 time} ]). Assume times=distances. Include reverse edges for bidirectional routes with same distance/time unless stated otherwise. Autocorrect minor typos in edge notation (example: 'a-c>:7' to 'a->c:7')
- Drones: List of dicts with name, capacity, energy (exp: [{"name": "drone1", "capacity": 1, "energy": 100}]). if no drone names are mentioned, use "drone1", "drone2", ...
- Optional Parameters: Dict with max_trips (per drone), t_max, w (landing/takeoff/service time), t_s (start time gap).
If parameters are missing, use defaults (max_trips=6 per drone, t_max=500, w=0, t_s=5) and list ambiguities in "missing_parameters" only if critical (exp: nodes, source, edges, drones).

Decision Variables:
- X_delivery[k,t,i,j]: Bool, if drone k uses edge (node i, node j) in trip t.
- start_time[k,t]: Int (>=0), start time of trip t for drone k. In constraints, denoted as s^k or start_time[k,t].
- arrival_time[k,t,i]: Int (>=0), arrival time at node i for drone k in trip t. In constraints, denoted as y_i^k or arrival_time[k,t,i].
- delivery_time[k,t,i]: Int (>=0), delivery time at node i for drone k in trip t (delivery nodes only). In constraints, denoted as y_i^k when referring to delivery times.
- resources_delivered[k,t,i]: Int (>=0, <= capacity), resources delivered to node i by drone k in trip t (delivery nodes only).
- load[k,i]: Int, resources carried by drone k at node i (positive at source, negative at delivery nodes, zero elsewhere).
- energy_used[k]: Int, total energy used by drone k.
- trip_active[k,t]: Bool, whether trip t for drone k is active.
- visited_nodes[k,t,i]: Bool, whether node i is visited by drone k in trip t.
- successor[k,t,i]: Int (-1 to len(nodes)-1), index of next node after i for drone k in trip t (-1 if not visited).
- drone_used[k]: Bool, whether drone k performs at least one trip.

The AI agent must be preprogrammed with the following base constraints. Make sure to encode these constraints in the output JSON under the "constraints" key using the suitable varaibles mentioned above.
Constraints:
#Core constraints
-Each drone trip must start at or after time zero.
-The arrival time at any node must always be non-negative.
-The starting times of two drones on the same trip must differ with a start time gap t_s.
-Each trip must begin with exactly one outgoing route from the source location.
-Deliveries made by different drones to the same location must be separated by at least the required waiting interval w(service time).
-When a drone travels directly from the source to another location, its arrival time must equal its start time plus the travel duration.
-When a drone travels between two delivery locations, its arrival time must equal the previous arrival time plus travel time and any waiting time (w).
-For every node, the number of incoming routes must equal the number of outgoing routes.
-Each visited node must have an arrival time less than or equal to the maximum allowed time t_max.
-A drone cannot travel from a node back to the same node.

#Resource and Load Constraints
```

```
-The total amount of resources delivered to each destination must exactly match that destination's demand.
-The total resources a drone delivers on a trip cannot exceed its carrying capacity.
-The total resources loaded at the source must equal the sum of all resources that will be delivered.
-Delivery nodes have negative loads equal to the amount they receive.
-Load is zero for every drone at any node that is neither the source nor a delivery node.
-The total amount of resources dispatched from the source cannot exceed what is available there.
-Every delivery quantity must be non-negative.

#Scheduling Constraints
-A trip must be active if there are outstanding delivery requirements.
-If no resources are delivered during a trip, that trip must be inactive and make no movements.
-Each trip must start after the previous one by at least the specified time gap
-If a delivery occurs at a node, the delivery time must equal the arrival time.
-If no delivery occurs, the delivery time must be zero.
-When a trip is active, the number of visited nodes must match the number of edges traversed.
-If a node is visited, its successor must correspond to an actual delivery edge (i to j) that the UAV travels.
-If the trip is active, the source node must also have a valid successor along a delivery edge (source to j).
-If a node is not visited, it must not have a successor (value = -1).
-If a trip is active, it must visit at least one delivery location
-If a trip is active, it must deliver at least one resource.
-The next trip must begin only after the previous trip has returned to the source.
-Every active trip must end by returning once to the source node.

#Energy Constraints
-The total energy used by a drone equals the sum of all distances based on its chosen routes.
-The total energy used by a drone must not exceed its available energy.

#Optimization Constraints
-Minimize the total energy used by all drones.
-Minimize the number of active trips right after.

For each constraint above, translate it into a Z3-readable logical expression string using decision variables and Z3 operators (exp: ForAll, Implies, And, Or, Sum, If).
Examples:
- 'Each drone trip must start at or after time zero.' to 'ForAll([k,t], start_time[k,t] >= 0)'
- 'The starting times of two drones on the same trip must differ with a start time gap t_s.' to 'ForAll([k1,k2,t], Implies(k1 != k2, Abs(start_time[k1,t] - start_time[k2,t]) >= {t_s}))'
- 'Each trip must begin with exactly one outgoing route from the source location.' to 'ForAll([k,t], Sum([If(X_delivery[k,t,source,j], 1, 0) for j in nodes]) == 1)'
-  Do not change the values for w or t_s or t_max in the constraints. Keep them as variables.
Use 'source' for source_node, 'nodes' for all nodes, 'delivery_nodes' for nodes with resources_required > 0.

Output a JSON object matching this schema:
{
  "nodes": [str],
  "source_node": str,
  "resources_available": {str: int},
  "resources_required": {str: int},
  "edges": [{"from": str, "to": str, "distance": int, "time": int}],
  "drones": [{"name": str, "capacity": int, "energy": int}],
  "optional": {"max_trips": int, "t_max": int, "w": int, "t_s": int},
  "missing_parameters": [str] or null,
  "constraints": [str]
}
```

Listing 1: Input Parsing Prompt with Constraints in Pseudocode

Model Variables: The variables listed below are declared from Line 29 to 39 within the Z3 model to represent UAV routing, timing, and resource delivery behavior. They correspond directly to the decision variables defined in Section 4.2, where the UAV network (N, E) and its operational parameters are introduced. These variables

are also declared in order to standardize the ensuring consistent interpretation and processing by the Z3 solver enabling proper enforcement of operational constraints and optimization objectives.
Below are the variables encoded representing a specific aspect of the model's logical and temporal structure:

- `X_delivery[k,t,i,j]` — Boolean variable corresponding to $x_{i,j}^{k,t}$. Used to indicate whether UAV k travels from node i to node j on trip t.
- `start_time[k,t]` — Integer variable corresponding to $s^{k,t}$, which represents the takeoff time of UAV k on trip t.
- `arrival_time[k,t,i]` — Corresponding to $y_i^{k,t}$, indicating the arrival (or service start) time of UAV k at node i.
- `delivery_time[k,t,i]` — Tracks the specific time at which resources are delivered to node i by UAV k on trip t.
- `resources_delivered[k,t,i]` — Corresponds to $r_i^{k,t}$, which denotes the resources quantity that UAV k delivers to node i on trip t.
- `load[k,i]` — Represents the current payload carried by UAV k at node i, used for ensuring delivery feasibility within capacity limits.
- `energy_used[k]` — Equivalent to bat^k, indicating the total energy consumed by UAV k across all trips.
- `trip_active[k,t]` — Boolean flag indicating whether UAV k is actively assigned to trip t.
- `visited_nodes[k,t,i]` — Boolean indicating whether if UAV k has visited node i during trip t.
- `successor[k,t,i]` — Stores the next node visited after node i by UAV k on trip t, contributing to route sequencing (π_k^t).
- `drone_used[k]` — Boolean variable denoting if UAV k is utilized in the current routing scenario.

Primary Model Constraints: The core routing and timing constraints in lines 34-43 corresponding to the constraints mentioned in Listing 1, define the basic logical and temporal framework of the UAV delivery model. The mathematical form of these constraints is meant to be converted into logical expressions that can be understood by Z3. Below, each constraint is described along with its role in ensuring feasible and safe UAV operations.

Line 44: This ensures each UAV's mission start time `start_time[k,t]` is non-negative, providing a valid temporal baseline for all flight operations.
Line 45: This ensures that all arrival or service start times `arrival_time[k,t,i]` at delivery nodes are greater than or equal to zero, ensuring temporal consistency across all nodes and UAVs.

Line 46: This enforces a minimum time gap `t_s` between the start times of any two UAVs `(k1, k2)` to prevent simultaneous take-off and ensure safe launch scheduling.
Line 47: This ensures that each UAV must begin its route from the source node with exactly one outgoing path, `sum_{j} X_delivery[k,t,source,j] = 1`.
Line 48: This ensures that if two UAVs are scheduled to deliver to the same node, their delivery times `delivery_time[k,t,i]` differ by at least the service duration `w`, preventing concurrent service at a single location.
Line 49: Defines the timing relationship between take-off time and delivery arrival time for UAVs departing directly from the source node. If `X_delivery[k,t,source,j]` is true, then `arrival_time[k,t,j] = start_time[k,t] + t[source,j]`, linking travel duration to temporal scheduling.
Line 50: Defines the timing relationship between consecutive nodes. When a UAV travels from node `i` to node `j`, the arrival time at `j` is calculated as `arrival_time[k,t,j] = arrival_time[k,t,i] + t[i,j] + w`, ensuring consistency in sequential timing throughout the route.
Line 51: Maintains route continuity by equating the number of incoming and outgoing routes at each node for each UAV, expressed as `sum_{j} X_delivery[k,t,i,j] = sum_{j} X_delivery[k,t,j,i]`, ensuring a consistent and connected delivery path.
Line 52: Enforces operational feasibility by ensuring that the arrival time at any node does not exceed the mission time limit `t_max`, thereby constraining deliveries within allowable time windows.
Line 53: Prevents UAVs from forming self-loops by enforcing `X_delivery[k,t,i,i] = False`, eliminating redundant movements from a node back to itself and preserving route efficiency.

Additional Constraints: Additional constraints specified in lines 56-85 were also added to ensure that the model enforces resource allocation, energy limits, and routing and timing logic ensuring consistency in how the problem is modeled. These are similarly defined in a mathematical format for translation into Z3 expressions. These additional constraints can be further categorized into Energy Constraints, Resource and Load Constraints, and Timing and Logical Constraints.

Resource and Load Constraints: Lines **56-62** are constraints that ensure the supply and demand are balanced while respecting the drones' resource capacity limits.
Line 56 ensures that the total resources delivered `resources_delivered[k,t,i]` across all UAVs and time steps exactly match the demand at each delivery node (`resources_required[i]`), guaranteeing that all recipients receive their required supplies.
Line 57 imposes a delivery capacity constraint, ensuring that no UAV exceeds its resource capacity (`drones[k].capacity`) during operation.
Line 58 defines the resources carried by each UAV at the source node (`load[k,source]`) as the total quantity of resources scheduled for delivery. This establishes the initial load state before the trip begins.
Line 59 Specifies the UAV's resources held at delivery nodes drops in proportion to the quantity of resources delivered. This is to ensure load conservation along the route.
Line 60 assigns zero load change for nodes that are neither source nor delivery nodes. This ensures that intermediate nodes visited do not alter the UAV's resources held.
Line 61 implies a resource supply constraint ensuring that the total resources deliv-

ered by all UAVs (`sum_k load[k,source]`) do not exceed the available resource supply(`resources_at[source]`.This will prevent the allocation of resources to exceed the available amount.
Line 62 Ensures that the quantity of resources delivered to any delivery node is non-negative.

Timing and Logical Constraints: The timing constraints are defined in Lines **65-77** and are used to govern how UAV trips are activated, how delivery and timing variables interact, and how logical relationships ensure consistent routing behavior throughout all trips.
Line 65: Ensures that at least one trip is activated (`trip_active[k,t]`) when pending delivery requests exist (`resources_required` non-empty), initiating UAV operations only when necessary.
Line 60: Limits each drone to at most `max_trips` trips, preventing unbounded mission schedules.
Line 61: Ensures that at least one resource unit during an active trip delivers(`sum_i resources_delivered[k,t,i] > 0`) preventing trivial trips.
Line 62: If no resources are delivered on a trip, the trip is inactive and all route variables are zero.
Line 63: Ensures that there is a minimum separation time (`t_s`) between the next trip of drone k, ensuring that each trip begins only after sufficient recovery time from the previous trip.
Line 64: If resources are delivered to node i by drone k in trip t, then the delivery time equals the arrival time: `delivery_time[k,t,i] = arrival_time[k,t,i]`.
Line 65: Ensures that if no resources are delivered to a delivery node i by drone k during trip t , then the delivery time at that node is set to zero preventing undefined delivery times when no delivery occurs.
Line 66: Ensures that the number of visited nodes equals the number of used edges for an active trip.
Line 67: If a non-source node is visited, there exists exactly one outgoing edge recorded in `successor[k,t,i]`.
Line 68: During an active trip t of drone k, the source node must have exactly one outgoing edge to a destination node j, recorded in `successor[k,t,source]`, ensuring the trip begins with a valid route.
Line 69: Nodes that are not visited have their successor set to `-1`.
Line 70: Every active trip must visit at least one delivery node.
Line 71: The start time of a drone k 's next trip $t+1$ cannot be earlier than its return to the source from trip t.
Line 72: Every active trip must end with exactly one incoming edge to the source node, closing the route.

Energy Constraints: Lines **80** and **81** define the UAVs' energy constraints, ensuring that the overall energy consumption of each UAV does not exceed its own battery capacity by adding up the distances of all the edges it travels. As a result, impractical assignments that would exhaust the drones' energy in the middle of a mission are avoided and directly tie selected paths to physical endurance limits.
Line 80 computes the total energy consumed by each UAV k as the sum of distances

along the traversed edges, where energy usage is accumulated only when a route segment $x_{i,j}^k$ is active.
Line 81 enforces that the calculated energy consumption for UAV k does not exceed its available battery capacity.

Optimization Constraint:
Optimization constraints mainly involve something thiosmge w **Line 84:** Minimize total energy consumption $\sum_k \texttt{energy_used}[k]$, then minimize total trips $\sum_{k,t} \texttt{trip_active}[k,t]$.

Z3 Translation Instructions: Lines **87-93** of the prompt provide instructions for translating the above constraints (lines 44-85) into Z3-readable logical expressions using operators such as `ForAll`, `Implies`, `Sum`, and `If`. These instructions specify that placeholders ({`t_s`}, {`w`}, {`t_max`}) should be substituted with values from the `optional` dictionary (or defaults: `max_trips=6`, `t_max=500`, `w=0`, `t_s=5`). Examples of translations to `ForAll([k,t], start_time[k,t] >= 0)`. The resulting constraints are to be included in the JSON output under the `constraints` key.

Required JSON output (the artefact passed to the solver LLM). The LLM must return exactly one JSON object that conforms to the schema below.

```
{
  "nodes": ["a", "b", "c", "d"],
  "source_node": "c",
  "resources_available": {"a": 10},
  "resources_required": {"b": 2, "c": 3, "d": 2},
  "edges": [
    {"from": "a", "to": "b", "distance": 3, "time": 3},
    {"from": "b", "to": "a", "distance": 3, "time": 3},
    {"from": "b", "to": "c", "distance": 4, "time": 4},
    {"from": "c", "to": "b", "distance": 4, "time": 4},
    {"from": "c", "to": "d", "distance": 5, "time": 5},
    {"from": "d", "to": "c", "distance": 5, "time": 5},
    {"from": "a", "to": "c", "distance": 7, "time": 7},
    {"from": "c", "to": "a", "distance": 7, "time": 7},
    {"from": "a", "to": "d", "distance": 12, "time": 12},
    {"from": "d", "to": "a", "distance": 12, "time": 12},
    {"from": "b", "to": "d", "distance": 9, "time": 9},
    {"from": "d", "to": "b", "distance": 9, "time": 9}
  ],
  "drones": [
    {"name": "drone1", "capacity": 1, "energy": 100},
    {"name": "drone2", "capacity": 2, "energy": 75}
  ],
  "optional": {"max_trips": 7, "t_max": 1000, "w": 5, "t_s": 4},
  "missing_parameters": null,
  "constraints": [
    "ForAll([k,t], start_time[k,t] >= 0)",
    "ForAll([k,t,i], arrival_time[k,t,i] >= 0)",
    "ForAll([k1,k2,t], Implies(k1 != k2, Abs(start_time[k1,t] - start_time[k2,t]) >= t_s))",
    "Minimize(Sum([energy_used[k] for k]))",
    "Minimize(Sum([If(trip_active[k,t], 1, 0) for k,t]))"
  ]
}
```

6.1.1 Comparison of Natural Language and Pseudocode Constraints

The test was ran using 2 different sets of constraints, one in natural language and one in pseudocode. In 2 the exact counterparts were used.

Table 2: Natural Language vs. Pseudocode Constraint Descriptions

No.	Natural Language Constraint	Pseudocode Constraint
1.Non-negative Start Time	Each drone trip must start at or after time zero.	**s**tart_time[k,t] $\geq$ 0 for all k, t
2.Non-negative Arrival Time	The arrival time at any node must always be non-negative.	**a**rrival_time[k,t,i] $\geq$ 0 for all i, k, t
3.Start Time Gap	The starting times of two drones on the same trip must differ with a start time gap t_s.	—start_time[k1,t] - start_time[k2,t]— $\geq$ {t_s} for k1 $\neq$ k2, t
4.Source Outgoing Route	Each trip must begin with exactly one outgoing route from the source location.	Implies(trip_active[k,t], sum_j X_delivery[k,t,source,j] = 1) for all k, t
5.Delivery Time Separation	Deliveries made by different drones to the same location must be separated by at least the required waiting interval w(service time).	If delivery_time[k1,t1,i] > 0 and delivery_time[k2,t2,i] > 0 then —delivery_time[k1,t1,i] - delivery_time[k2,t2,i]— $\geq$ {w} for all k1,k2,t1,t2,i in delivery_nodes (k1,t1) $\neq$ (k2,t2)
6.Source-to-Node Arrival:	When a drone travels directly from the source to another location, its arrival time must equal its start time plus the travel duration.	Implies(X_delivery[k,t,source,j] = 1, arrival_time[k,t,j] = start_time[k,t] + t_source,j) for all k, t, j
7.Node-to-Node Arrival:	When a drone travels between two delivery locations, its arrival time must equal the previous arrival time plus travel time and any waiting time (w).	Implies(X_delivery[k,t,i,j] = 1, arrival_time[k,t,j] = arrival_time[k,t,i] + t_i,j + {w}) for all k, t, i in delivery_nodes, j
8.Flow Conservation:	For every node, the number of incoming routes must equal the number of outgoing routes.	sum_j X_delivery[k,t,i,j] = sum_j X_delivery[k,t,j,i] for all i, k, t
9.Max Arrival Time:	Each visited node must have an arrival time less than or equal to the maximum allowed time t_max.	Implies(sum_j X_delivery[k,t,i,j] > 0, arrival_time[k,t,i] $\leq$ {t_max}) for all i, k, t
10.No Self-Loops:	A drone cannot travel from a node back to the same node.	X_delivery[k,t,i,i] = 0 for all i, k, t
11.Resource Delivery Balance:	The total amount of resources delivered to each destination must exactly match that destination's demand.	sum_{k,t} resources_delivered[k,t,i] = resources_required[i] for all i in delivery_nodes
12.Resource Capacity Limit:	The total resources a drone delivers on a trip cannot exceed its carrying capacity.	sum_i resources_delivered[k,t,i] $\leq$ drones[k]['capacity'] for all k, t
13.Source Load Balance:	The total resources loaded at the source must equal the sum of all resources that will be delivered.	load[k,source] = sum_{t,i in delivery_nodes} resources_delivered[k,t,i] for all k
14.Delivery Node Load:	Delivery nodes have negative loads equal to the amount they receive.	load[k,i] = -sum_t resources_delivered[k,t,i] for all k, i in delivery_nodes
15.Non-Delivery Node Load:	Load is zero for every drone at any node that is neither the source nor a delivery node.	load[k,i] = 0 for all k, i not in {source, delivery_nodes}
16.Source Resource Limit:	The total amount of resources dispatched from the source cannot exceed what is available there.	sum_k load[k,source] $\leq$ resources_at[source]
17.Non-negative Delivery Amount:	Every delivery quantity must be non-negative.	resources_delivered[k,t,i] $\geq$ 0 for all k, t, i in delivery_nodes

Continued on next page

Table 2: Natural Language vs. Pseudocode Constraint Descriptions (Continued)

No.	Natural Language Constraint	Pseudocode Constraint
18.Trip Activation:	A trip must be active if there are outstanding delivery requirements.	If sum(resources_required[i] for i in delivery_nodes) > 0 then Or(trip_active[k,t] for k, t)
19.No Delivery Implies Inactive Trip:	If no resources are delivered during a trip, that trip must be inactive and make no movements.	Implies(sum_i resources_delivered[k,t,i] = 0, And(Not(trip_active[k,t]), sum_{i,j} X_delivery[k,t,i,j] = 0)) for all k, t
20.Trip Time Gap:	Each trip must start after the previous one by at least the specified time gap.	start_time[k,t+1] ≥ start_time[k,t] + {t_s} for all k, t < max_trips-1
21.Delivery Time Equals Arrival:	If a delivery occurs at a node, the delivery time must equal the arrival time.	Implies(resources_delivered[k,t,i] > 0, delivery_time[k,t,i] = arrival_time[k,t,i]) for all k, t, i in delivery_nodes
22. No Delivery, Zero Time:	If no delivery occurs, the delivery time must be zero.	Implies(resources_delivered[k,t,i] = 0, delivery_time[k,t,i] = 0) for all k, t, i in delivery_nodes
23.Visited Node Equals Traversed Edges:	When a trip is active, the number of visited nodes must match the number of edges traversed.	Implies(trip_active[k,t], sum_i visited_nodes[k,t,i] = sum_{i,j} If(X_delivery[k,t,i,j], 1, 0)) for all k, t
24.Successor for Visited Nodes:	If a node is visited, its successor must correspond to an actual delivery edge (i to j) that the UAV travels.	Implies(visited_nodes[k,t,i], Or(And(X_delivery[k,t,i,j], successor[k,t,i] = node_indices[j]) for j where (i,j) in edges)) for all k, t, i ≠ source
25.Source Successor:	If the trip is active, the source node must also have a valid successor along a delivery edge (source to j).	Implies(trip_active[k,t], Or(And(X_delivery[k,t,source,j], successor[k,t,source] = node_indices[j]) for j where (source,j) in edges)) for all k, t
26.No Successor for Unvisited Nodes:	If a node is not visited, it must not have a successor (value = -1).	Implies(Not(visited_nodes[k,t,i]), successor[k,t,i] = -1) for all k, t, i
27.Active Trip Visit Delivery:	If a trip is active, it must visit at least one delivery location.	Implies(trip_active[k,t], sum_i resources_delivered[k,t,i] > 0) for all k, t
28.Active Trip Deliver Resource:	If a trip is active, it must deliver at least one resource.	Implies(trip_active[k,t], sum_{i in delivery_nodes} visited_nodes[k,t,i] ≥ 1) for all k, t
29.Sequential Trips:	The next trip must begin only after the previous trip has returned to the source.	Implies(trip_active[k,t], start_time[k,t+1] ≥ arrival_time[k,t,source]) for all k, t < max_trips-1
30.Return to Source:	Every active trip must end by returning once to the source node.	Implies(trip_active[k,t], sum_j If(X_delivery[k,t,j,source], 1, 0) = 1) for all k, t
31.Energy Usage Calculation	The total energy used by a drone equals the sum of all distances based on its chosen routes.	energy_used[k] = sum_{t,i,j} If(X_delivery[k,t,i,j], edges[(i,j)]['distance'], 0) for all k
32.Energy Capacity Limit	The total energy used by a drone must not exceed its available energy.	energy_used[k] ≤ drones[k]['energy'] for all k

Continued on next page

Table 2: Natural Language vs. Pseudocode Constraint Descriptions (Continued)

No.	Natural Language Constraint	Pseudocode Constraint
33. Minimize Total Energy	Minimize the total energy used by all drones.	Minimize sum_k energy_used[k]
34.Minimize Active Trips	Minimize the number of active trips right after.	Then minimize sum_{k,t} trip_active[k,t]

6.2 User Prompt

User Prompt

I got 5 nodes, $'a', 'b', 'c', 'd', 'e'$. The starting point is node $'c'$. The distance are as follows, $a \rightarrow b : 3$, $b \rightarrow c : 4$, $c \rightarrow d : 5$, $d \rightarrow e : 3$, $c \rightarrow e : 8$, $a - c >: 7$, $a \rightarrow d : 12$, $a \rightarrow e : 15$, $b \rightarrow d : 9$, $b \rightarrow e : 12$ The edge distances are the same if they are reversed. Node 'c' has 15 resources. The resources needed in each node are $b : 2, a : 3, d : 2, e : 1$. There are 3 drones, drone1 has 1 storage capacity and 100 energy, drone2 has 2 storage capacity and 75 energy, drone3 has 3 storage capacity and 50 energy. The maximum allowed time is 1000 units time, maximum trips are 7. Include service time which is 5 unit time, take off time is 5.

The model's performance and constraint enforcement were verified using a moderately sized test instance that provides a compromise between operational complexity and computational viability. The scenario has three drones with different capacities and energy limits, five nodes (four delivery nodes and one source node), and a total demand of 8 resource units among the delivery nodes. A minor typo was included to test the the LLM's ability to correct a simple error made by a user. This problem size is suitable for evaluating essential features like temporal scheduling, resource allocation, and energy minimization without requiring an unreasonably long runtime because it is neither absurdly large (allowing for rapid iteration and debugging) nor trivial (allowing meaningful multi-drone coordination and route optimization).

6.3 Code Generation

Configuration: The AI agent used will be GPT-5 model from OpenAI that will parse natural language disaster scenarios as input and code generation. For formal verification, the Z3 SMT solver is encoded ..... The AI agent used will be GPT-4o
During the verification process, the Z3's *check*() method will determine if the constraints are satisfiable(*sat*). If so, the Z3 solver will produce a model which can satisfy the constraints and provide a valid UAV schedule and resource allocation plan. Should the problem not be satisfiable(*unsat*), Z3's unsat core identifies conflicting constraints, and the LLM will suggests parameter adjustments such as increasing drone energy, capacity or increase resources based on the analysis. The Iterative solving optimizes the energy and trip counters within constraints.

Solution Extraction:If the solver returns sat, the model is evaluated to extract:

- Routes: Paths π_k derived from $x^k_{t,i,j} = 1$.
- Schedules: Takeoff times s^k_t and delivery times $y^k_{t,i}$.
- Resource Allocations: Resources $r^k_{t,i}$ delivered per node.
- Energy Usage: Total bat^k per UAV.

Solution Evaluation: Solutions are evaluated for feasibility (all constraints satisfied) and optimality (minimum energy or trips). If infeasible, Z3's unsat core identifies conflicting constraints. The LLM analyzes the unsat core to suggest parameter adjustments, such as increasing drone energy or relaxing time constraints.

Prompt Design: The system prompt is designed to guide the LLM (GPT-5) to generate a function, `run_z3_solver(params)`, which models the UAV routing problem using input parameters and generated constraints from the JSON file. This function incorporates the decision variables defined in [**?**] and applies the constraints within the Z3 solver by parsing and encoding them using logical operators such as `ForAll`, `Implies`, and `Sum`, substituting parameters (example: `source` with `params['initial_node']`) and encoding them via `add()`. The Z3 is then used to determine if the problem is satisfiable under these constraints to produce a mission plan for the given scenario. An output of either satisfiable or unsatisfiable will be given by Z3. For satisfiable solutions, it will generate a detailed mission plan for each drone, including energy used, trip count, and a tabular summary of paths, start/end times, delivery times, and resources delivered. A nested function, `find_path(k,t)`, is then used to reconstruct the valid routes for each drone k and trip t, from the solver's output developing the mission plan.

System Prompt

```
You are an expert Python developer and Z3 modeler. Generate a Python function "def run_z3_solver(params):" to solve a drone routing and delivery optimization problem using Z3.
The function models the UAV routing problem by directly encoding all constraints in Z3, without parsing a constraints list. It returns a string with the solution or unsatisfiability reason.

Problem Description:
- Drones deliver resources from a source node to delivery nodes in a weighted, bidirectional graph.
- Each drone has capacity and energy limits, follows edges with distance/time costs, and must return to the source after each trip.
- Constraints ensure valid paths, resource delivery, energy limits, and timing (exp, service time 'w', start time gap 't_s').

Inputs (params dict):
- 'nodes': list[str] (exp, ["a", "b", "c"])
- 'initial_node': str (exp, "c")
- 'resources_at': dict[str,int] (exp, {"c": 15})
- 'resources_required': dict[str,int] (exp, {"a": 3, "b": 2})
- 'edges': dict[(str,str)]->dict (exp, {("a","b"): {"distance": 3, "time": 3}})
- 'drones': dict[str]->dict (exp, {"drone1": {"capacity": 1, "energy": 100}})
- 'optional': dict with 'max_trips' (default 6), 't_max' (default 500), 'w' (default 4), 't_s' (default 5)
- 'delivery_nodes': list[str] (nodes with resources_required > 0)
(Use default values if optional params are missing from JSON file.)
Decision Variables:
- X_delivery[k,t,i,j]: Bool, drone k uses edge (i,j) in trip t.
- start_time[k,t]: Int (>=0), start time of trip t for drone k.
- arrival_time[k,t,i]: Int (>=0), arrival time at node i for drone k in trip t.
```

```
- delivery_time[k,t,i]: Int (>=0), delivery time at node i for drone k in trip t (delivery nodes only).
- resources_delivered[k,t,i]: Int (>=0, <= capacity), resources delivered to node i by drone k in trip t.
- load[k,i]: Int, resources carried by drone k at node i.
- energy_used[k]: Int, total energy used by drone k.
- trip_active[k,t]: Bool, whether trip t for drone k is active.
- visited_nodes[k,t,i]: Bool, whether node i is visited by drone k in trip t.
- successor[k,t,i]: Int (-1 to len(nodes)-1), index of next node after i for drone k in trip t.
- drone_used[k]: Bool, whether drone k performs any trips.

Constraints:
You must formally model the Constraint Satisfaction Problem by encoding the constraints from params['constraints']. For each constraint string:
1. - Identify iteration variables and domains:
     - 'k': 'drones.keys()'
     - 't': 'range(max_trips)'
     - 'i', 'j': 'nodes' or 'delivery_nodes' (if "i in delivery_nodes")
   - Substitute placeholders:
     - 'source' -> 'params['initial_node']'
     - 'nodes' -> 'params['nodes']'
     - 'delivery_nodes' -> 'params['delivery_nodes']'
     - 'drones' -> 'drones.keys()'
     - 'max_trips', 'w', 't_s', 't_max' -> 'params['optional']' values
     - 'edges[(i,j)]['time']' -> 'edges[(i,j)]['time']'
     - 'node_indices[j]' -> 'node_indices[j]' (precompute 'node_indices = {n: idx for idx, n in enumerate(nodes)}')
2. - Implement into Z3:
   - Use evaluate():
   - Translate each constraint into a Z3 expression using 'ForAll', 'Implies', 'And', 'Or', 'Sum', 'If', etc.
   - Handle complex constraints (exp, 'ForAll([k,t,i], Implies(And(i != source, visited_nodes[k,t,i]), Or([And(X_delivery[k,t,i,j], successor[k,t,i] == node_indices[j]) for j in nodes if (i,j) in edges])))') by iterating over appropriate domains and constructing 'Or' conditions.
3. - Ensure Completeness:
   - Process each constraint in params['constraints']. If a pattern from the constraint is unrecognized, log a warning and continue with the rest. Ensure core constraints are not skipped.
   - Ensure decision variables are logically linked through the constraints to reflect route consistency (exp, 'visited_nodes[k,t,i]' is true if any edge '(i,j)' or '(j,i)' is used).

Path Finding
- Implement 'find_path(k,t)' to construct a path starting at 'initial_node', following 'X_delivery[k,t,i,j]', and returning to 'initial_node' if 'trip_active[k,t]'. Limit to 'len(nodes)*2' steps to prevent infinite loops.
- Validate that active trips have non-empty paths and visit all nodes where 'resources_delivered[k,t,i] > 0'.
- Return list of edges '[(i,j), ...]' or '[]' if no valid path or trip inactive.

Output Format:
If satisfiable:
- "Satisfiable\nSolve time: {time:.2f} seconds\nBest Solution:"
- For each drone k (sorted):
  - If no trips: "Drone {k}:\n  No delivery trips"
  - Else:
   Format the solver output exactly like this example:
    Drone drone3:
    Energy used: 38
    Number of delivery trips: 3

    Trip Summary:
    Trip  Path                          Start  End    Delivery Times       EN Used Current EN Resources Held Required Amount

    ------------------------------------------------------------------------------------------------------------------------------

    1     c->b, b->c                    0      13     b@4                  8       42         2
                  b(0), a(3), d(2), e(1)
    2     c->b, b->a, a->c              25     49     a@37                 14      28         3
```

```
                    b(0), a(0), d(2), e(1)
    3     c->e, e->d, d->c               53     79     d@69, e@61           16      12          3
                    b(0), a(0), d(0), e(0)

    Then print:
    Overall summary:
  Total resources delivered: {'b': 2, 'a': 3, 'd': 2, 'e': 1}
  Total energy used: 38
  Total trips: 3
  Resources required: {'b': 2, 'a': 3, 'd': 2, 'e': 1}
If unsatisfiable:
- "Unsatisfiable\nReason: {core or error}\nSolve time: {time:.2f} seconds"

Implementation
- Treat the problem as a CSP where all constraints in 'params['constraints']' must be satisfied.
    Translate each constraint into Z3 expressions accurately, preserving their logical structure
    .
- Use string parsing (exp, regex or tokenization) to interpret constraint strings, identifying
    variables, operators ('ForAll', 'Implies', etc.), and domains. Map these to Z3 constructs
    dynamically.
- Use int values, bound loads (-50..50), successor (-1..len(nodes)-1).
- Catch exceptions: Return "Unsatisfiable\nReason: Solver crashed: {e}\nSolve time: ...".
- No external services; use z3, standard library.
- Test with: 5 nodes (a,b,c,d,e), depot c, bidirectional edges, drones cap1-3/energy50-100, req
    a3/b2/d2/e1, max_trips=7/t_max=5000/w=5/t_s=4.
Return only the Python function def run_z3_solver(params): ...
"""
```

Listing 2: Code Generation Prompt for Z3 Solver Implementation

7 Discussion

7.1 Comparison of Natural Language and Pseudocode Constraints

To assess how well the LLM translates constraints for the UAV routing model, two approaches were tested : natural language descriptions and pseudocode for mathematical modeling. Despite similarities in the high-level decision variables and problem structure, the different constraint explanations resulted variations in the generated Z3 expressions leading to distinct solver outputs and feasibility outcomes.

- **Natural Language**: Constraints were described in plain English, making them accessible to non-technical users like disaster managers. However, vague phrasing sometimes can lead to ambiguous Z3 translations, resulting in feasible but impractical solutions (example: zero deliveries).

- **Pseudocode**: Constraints were written in precise, mathematical notation, closely matching Z3 syntax. This produced accurate Z3 expressions, yielding complete and practical solutions but requiring more technical expertise as minor misunderstandings could also results on .

The following table includes the including nodes, resources, edges, drones, and optional parameters used in the experiments. In this scenario, the optional parameters are set as $t_s = 4$, $t_max = 5000$, and $w = 5$, defining the start time gap, maximum allowed time, and waiting interval, respectively.

The variables used for the test are as below.

Parameter	Details
Nodes	a, b, c, d, e
Source Node	c
Resources Available	c: 15
Resources Required	b: 2, a: 3, d: 2, e: 1
Edges	$a \to b$: 3, 3; $b \to a$: 3, 3; $b \to c$: 4, 4; $c \to b$: 4, 4; $c \to d$: 5, 5; $d \to c$: 5, 5; $d \to e$: 3, 3; $e \to d$: 3, 3; $a \to c$: 7, 7; $c \to a$: 7, 7; $c \to e$: 8, 8; $e \to c$: 8, 8; $a \to d$: 12, 12; $d \to a$: 12, 12; $a \to e$: 15, 15; $e \to a$: 15, 15; $b \to d$: 9, 9; $d \to b$: 9, 9; $b \to e$: 12, 12; $e \to b$: 12, 12
Drones	drone1: capacity 1, energy 100; drone2: capacity 2, energy 75; drone3: capacity 3, energy 50
Optional Parameters	max_trips: 7, t_max: 5000, w: 5, t_s: 4

Table 3: Parameters used for experiment

7.1.1 Generated Z3 Constraints from Natural Language

Table 4: Z3 Constraint Generation

Constraint	Test 1	Test 2	Test 3
1.Non-negative Start Time:	`ForAll([k,t], start_time[k,t] >= 0)`	`ForAll([k,t], start_time[k,t] >= 0)`	`ForAll([k,t], start_time[k,t] >= 0)`
2.Non-negative Arrival Time:	`ForAll([k,t,i], arrival_time[k,t,i] >= 0)`	`ForAll([k,t,i], arrival_time[k,t,i] >= 0)`	`ForAll([k,t,i], arrival_time[k,t,i] >= 0)`
3.Start Time Gap:	`ForAll([k1,k2,t], Implies(k1 != k2, Abs(start_time[k1,t] - start_time[k2,t]) >= t_s))`	`ForAll([k1,k2,t], Implies(k1 != k2, Abs(start_time[k1,t] - start_time[k2,t]) >= t_s))`	`ForAll([k1,k2,t], Implies(k1 != k2, Abs(start_time[k1,t] - start_time[k2,t]) >= t_s))`
4.Source Outgoing Route:	`ForAll([k,t], Implies(trip_active[k,t], Sum([If(X_delivery[k,t,source,j], 1, 0) for j in nodes]) == 1))`	`ForAll([k,t], Sum([If(X_delivery[k,t,source,j], 1, 0) for j in nodes]) == If(trip_active[k,t], 1, 0))`	`ForAll([k,t], Sum([If(X_delivery[k,t,source,j], 1, 0) for j in nodes if j != source]) == If(trip_active[k,t], 1, 0))`
5.Delivery Time Separation:	`ForAll([k1,k2,t1,t2,i], Implies(And(k1 != k2, i in delivery_nodes, resources_delivered[k1,t1,i] > 0, resources_delivered[k2,t2,i] > 0), Abs(arrival_time[k1,t1,i] - arrival_time[k2,t2,i]) >= w))`	`ForAll([k1,k2,t1,t2,i], Implies(And(k1 != k2, i in delivery_nodes, resources_delivered[k1,t1,i] > 0, resources_delivered[k2,t2,i] > 0), Abs(delivery_time[k1,t1,i] - delivery_time[k2,t2,i]) >= w))`	`ForAll([k1,k2,t1,t2,i], Implies(And(Or(k1 != k2, t1 != t2), resources_delivered[k1,t1,i] > 0, resources_delivered[k2,t2,i] > 0), Abs(delivery_time[k1,t1,i] - delivery_time[k2,t2,i]) >= w))`
6.Source-to-Node Arrival:	`ForAll([k,t,j], Implies(And(j != source, X_delivery[k,t,source,j]), arrival_time[k,t,j] == start_time[k,t] + time(source,j)))`	`ForAll([k,t,j], Implies(X_delivery[k,t,source,j], arrival_time[k,t,j] == start_time[k,t] + dist(source,j)))`	`ForAll([k,t,j], Implies(X_delivery[k,t,source,j], arrival_time[k,t,j] == start_time[k,t] + time[source,j]))`

Continued on next page

Table 4: Z3 Constraint Generation

Constraint	Test 1	Test 2	Test 3
7.Node-to-Node Arrival:	`ForAll([k,t,i,j], Implies(And(i in delivery_nodes, j in delivery_nodes, i != j, X_delivery[k,t,i,j]), arrival_time[k,t,j] == arrival_time[k,t,i] + time(i,j) + w))`	`ForAll([k,t,i,j], Implies(And(X_delivery[k,t,i,j], i != source), arrival_time[k,t,j] == arrival_time[k,t,i] + dist(i,j) + If(resources_delivered[k,t,i] > 0, w, 0)))`	`ForAll([k,t,i,j], Implies(And(i != source, j != source, X_delivery[k,t,i,j]), arrival_time[k,t,j] == arrival_time[k,t,i] + w + time[i,j]))`
8.Flow Conservation:	`ForAll([k,t,i], Sum([If(X_delivery[k,t,i,j], 1, 0) for j in nodes]) == Sum([If(X_delivery[k,t,j,i], 1, 0) for j in nodes]))`	`ForAll([k,t,i], Sum([If(X_delivery[k,t,j,i], 1, 0) for j in nodes]) == Sum([If(X_delivery[k,t,i,j], 1, 0) for j in nodes]))`	`ForAll([k,t,i], Sum([If(X_delivery[k,t,j,i], 1, 0) for j in nodes]) == Sum([If(X_delivery[k,t,i,j], 1, 0) for j in nodes]))`
9.Max Arrival Time:	`ForAll([k,t,i], Implies(visited_nodes[k,t,i], arrival_time[k,t,i] <= t_max))`	`ForAll([k,t,i], Implies(visited_nodes[k,t,i], arrival_time[k,t,i] <= t_max))`	`ForAll([k,t,i], Implies(visited_nodes[k,t,i], arrival_time[k,t,i] <= t_max))`
10.No Self-Loops:	`ForAll([k,t,i], Not(X_delivery[k,t,i,i]))`	`ForAll([k,t,i], Not(X_delivery[k,t,i,i]))`	`ForAll([k,t,i], Not(X_delivery[k,t,i,i]))`
11.Resource Delivery Balance:	`ForAll([i], Implies(i in delivery_nodes, Sum([resources_delivered[k,t,i] for k in drones for t in Trips]) == resources_required[i]))`	`ForAll([i], Implies(i in delivery_nodes, Sum([resources_delivered[k,t,i] for k in drones for t in range(max_trips)]) == resources_required[i]))`	`ForAll([i], Implies(member(i, delivery_nodes), Sum([resources_delivered[k,t,i] for k,t]) == resources_required[i]))`
12.Resource Capacity Limit:	`ForAll([k,t], Sum([resources_delivered[k,t,i] for i in delivery_nodes]) <= capacity[k])`	`ForAll([k,t], Sum([resources_delivered[k,t,i] for i in delivery_nodes]) <= capacity[k])`	`ForAll([k,t], Sum([resources_delivered[k,t,i] for i in delivery_nodes]) <= capacity[k])`
13.Source Load Balance:	`ForAll([k], load[k,source] == Sum([resources_delivered[k,t,i] for t in Trips for i in delivery_nodes]))`	`Sum([load[k,source] for k in drones]) == Sum([resources_delivered[k,t,i] for k in drones for t in range(max_trips) for i in delivery_nodes])`	`Sum([load[k,source] for k]) == Sum([resources_required[i] for i in delivery_nodes])`
14.Delivery Node Load:	`ForAll([k,i], Implies(i in delivery_nodes, load[k,i] == -Sum([resources_delivered[k,t,i] for t in Trips])))`	`ForAll([k,i], Implies(i in delivery_nodes, load[k,i] == -Sum([resources_delivered[k,t,i] for t in range(max_trips)])))`	`ForAll([k,i], Implies(member(i, delivery_nodes), load[k,i] == -Sum([resources_delivered[k,t,i] for t])))`
15.Non-Delivery Node Load:	`ForAll([k,i], Implies(And(i != source, Not(i in delivery_nodes)), load[k,i] == 0))`	`ForAll([k,i], Implies(And(i != source, Not(i in delivery_nodes)), load[k,i] == 0))`	`ForAll([k,i], Implies(And(i != source, Not(member(i, delivery_nodes))), load[k,i] == 0))`
16.Source Resource Limit:	`Sum([load[k,source] for k in drones]) <= resources_available[source]`	`Sum([load[k,source] for k in drones]) <= resources_available[source]`	`Sum([load[k,source] for k]) <= resources_available[source]`
17.Non-negative Delivery Amount:	`ForAll([k,t,i], resources_delivered[k,t,i] >= 0)`	`ForAll([k,t,i], Implies(i in delivery_nodes, resources_delivered[k,t,i] >= 0))`	`ForAll([k,t,i], resources_delivered[k,t,i] >= 0)`
18.Trip Activation:	`ForAll([k,t], Implies(Sum([resources_delivered[k,t,i] for i in delivery_nodes]) > 0, trip_active[k,t]))`	`ForAll([k,t], Implies(Sum([resources_delivered[k,t,i] for i in delivery_nodes]) > 0, trip_active[k,t]))`	`ForAll([k,t], If(Sum([resources_delivered[k,t,i] for i in delivery_nodes]) > 0, trip_active[k,t], Not(trip_active[k,t])))`

Continued on next page

Table 4: Z3 Constraint Generation

Constraint	Test 1	Test 2	Test 3
19.No Delivery Implies Inactive Trip:	`ForAll([k,t], Implies(Sum([resources_delivered[k,t,i] for i in delivery_nodes]) == 0, And(Not(trip_active[k,t]), Sum([If(X_delivery[k,t,i,j], 1, 0) for i in nodes for j in nodes]) == 0)))`	`ForAll([k,t], Implies(Sum([resources_delivered[k,t,i] for i in delivery_nodes]) == 0, And(Not(trip_active[k,t]), Sum([If(X_delivery[k,t,i,j], 1, 0) for i in nodes for j in nodes]) == 0)))`	`ForAll([k,t], Implies(Not(trip_active[k,t]), And(Sum([If(X_delivery[k,t,i,j], 1, 0) for i,j in nodes_pairs(nodes)]) == 0, Sum([If(visited_nodes[k,t,i], 1, 0) for i in nodes]) == 0)))`
20.Trip Time Gap:	`ForAll([k,t], Implies(t > 0, start_time[k,t] >= start_time[k,t-1] + t_s))`	`ForAll([k,t], Implies(t > 0, start_time[k,t] >= start_time[k,t-1] + t_s))`	`ForAll([k,t], Implies(t > 0, start_time[k,t] >= start_time[k,t-1] + t_s))`
21.Delivery Time Equals Arrival:	`ForAll([k,t,i], Implies(resources_delivered[k,t,i] > 0, delivery_time[k,t,i] == arrival_time[k,t,i]))`	`ForAll([k,t,i], Implies(resources_delivered[k,t,i] > 0, delivery_time[k,t,i] == arrival_time[k,t,i]))`	`ForAll([k,t,i], If(resources_delivered[k,t,i] > 0, delivery_time[k,t,i] == arrival_time[k,t,i], delivery_time[k,t,i] == 0))`
22.No Delivery, Zero Time:	`ForAll([k,t,i], Implies(resources_delivered[k,t,i] == 0, delivery_time[k,t,i] == 0))`	`ForAll([k,t,i], Implies(resources_delivered[k,t,i] == 0, delivery_time[k,t,i] == 0))`	`ForAll([k,t,i], If(resources_delivered[k,t,i] > 0, delivery_time[k,t,i] == arrival_time[k,t,i], delivery_time[k,t,i] == 0))`
23.Visited Node Equals Traversed Edges:	`ForAll([k,t], Implies(trip_active[k,t], Sum([If(visited_nodes[k,t,i], 1, 0) for i in nodes]) == Sum([If(X_delivery[k,t,i,j], 1, 0) for i in nodes for j in nodes])))`	`ForAll([k,t], Implies(trip_active[k,t], Sum([If(visited_nodes[k,t,i], 1, 0) for i in nodes]) == Sum([If(X_delivery[k,t,i,j], 1, 0) for i in nodes for j in nodes])))`	`ForAll([k,t], Implies(trip_active[k,t], Sum([If(visited_nodes[k,t,i], 1, 0) for i in nodes]) == Sum([If(X_delivery[k,t,i,j], 1, 0) for i,j in nodes_pairs(nodes)])))`
24.Successor for Visited Nodes:	`ForAll([k,t,i], Implies(visited_nodes[k,t,i], And(successor[k,t,i] >= 0, successor[k,t,i] < Len(nodes), X_delivery[k,t,i, nodes[successor[k,t,i]]])))`	`ForAll([k,t,i], Implies(visited_nodes[k,t,i], Exists([j], And(j >= 0, j < Len(nodes), successor[k,t,i] == j, X_delivery[k,t,i,nodes[j]])))))`	`ForAll([k,t,i], Implies(visited_nodes[k,t,i], And(successor[k,t,i] >= -1, successor[k,t,i] < len(nodes), Or([And(successor[k,t,i] == index(j), X_delivery[k,t,i,j]) for j in nodes]))))`
25.Source Successor:	Covered by **Successor for Visited Nodes(24)**	`ForAll([k,t], Implies(trip_active[k,t], Exists([j], And(j >= 0, j < Len(nodes), X_delivery[k,t,source,nodes[j]], successor[k,t,source] == j))))`	`ForAll([k,t], Implies(trip_active[k,t], And(successor[k,t,source] >= 0, successor[k,t,source] < len(nodes), X_delivery[k,t,source,nodes[successor[k,t,s`
26.No Successor for Unvisited Nodes:	`ForAll([k,t,i], Implies(Not(visited_nodes[k,t,i]), successor[k,t,i] == -1))`	`ForAll([k,t,i], Implies(Not(visited_nodes[k,t,i]), successor[k,t,i] == -1))`	`ForAll([k,t,i], Implies(Not(visited_nodes[k,t,i]), successor[k,t,i] == -1))`
27.Active Trip Visit Delivery:	`ForAll([k,t], Implies(trip_active[k,t], Sum([If(visited_nodes[k,t,i], 1, 0) for i in delivery_nodes]) >= 1))`	`ForAll([k,t], Implies(trip_active[k,t], Sum([If(visited_nodes[k,t,i], 1, 0) for i in delivery_nodes]) >= 1))`	`ForAll([k,t], Implies(trip_active[k,t], Sum([If(visited_nodes[k,t,i], 1, 0) for i in delivery_nodes]) >= 1))`
28.Active Trip Deliver Resource:	`ForAll([k,t], Implies(trip_active[k,t], Sum([resources_delivered[k,t,i] for i in delivery_nodes]) >= 1))`	`ForAll([k,t], Implies(trip_active[k,t], Sum([resources_delivered[k,t,i] for i in delivery_nodes]) >= 1))`	|
29.Sequential Trips:	`ForAll([k,t], Implies(t > 0, start_time[k,t] >= arrival_time[k,t-1,source]))`	`ForAll([k,t], Implies(t > 0, start_time[k,t] >= arrival_time[k,t-1,source]))`	`ForAll([k,t], Implies(t > 0, start_time[k,t] >= arrival_time[k,t-1,source] + t_s))`

Continued on next page

Table 4: Z3 Constraint Generation

Constraint	Test 1	Test 2	Test 3
30.Return to Source:	`ForAll([k,t], Implies(trip_active[k,t], Sum([If(X_delivery[k,t,i,source], 1, 0) for i in nodes]) == 1))`	`ForAll([k,t], Implies(trip_active[k,t], Sum([If(X_delivery[k,t,i,source], 1, 0) for i in nodes]) == 1))`	`ForAll([k,t], Sum([If(X_delivery[k,t,i,source], 1, 0) for i in nodes if i != source]) == If(trip_active[k,t], 1, 0))`
31.Energy Usage Calculation:	`ForAll([k], energy_used[k] == Sum([If(X_delivery[k,t,i,j], distance(i,j), 0) for t in Trips for i in nodes for j in nodes]))`	`ForAll([k], energy_used[k] == Sum([If(X_delivery[k,t,i,j], dist(i,j), 0) for t in range(max_trips) for i in nodes for j in nodes]))`	`ForAll([k], energy_used[k] == Sum([If(X_delivery[k,t,i,j], distance[i,j], 0) for t,i,j]))`
32.Energy Capacity Limit:	`ForAll([k], energy_used[k] <= energy[k])`	`ForAll([k], energy_used[k] <= energy[k])`	`ForAll([k], energy_used[k] <= energy[k])`
33.Minimize Total Energy:	`Minimize(Sum([energy_used[k] for k in drones]))`	`Minimize(Sum([energy_used[k] for k in drones]))`	`Minimize(Sum([energy_used[k] for k]))`
34.Minimize Active Trips:	`Minimize(Sum([If(trip_active[k,t], 1, 0) for k in drones for t in Trips]))`	`Minimize(Sum([If(trip_active[k,t], 1, 0) for k in drones for t in range(max_trips)]))`	`Minimize(Sum([If(trip_active[k,t], 1, 0) for k,t in KT]))`
35.Drone Usage Indicator (Extra):	`ForAll([k], drone_used[k] == Or([trip_active[k,t] for t in Trips]))`	`ForAll([k], drone_used[k] == Or([trip_active[k,t] for t in range(max_trips)]))`	`ForAll([k], drone_used[k] == Or([trip_active[k,t] for t in T]))`

Analysis: Across the three Tests to generate Z3 constraints from natural language specifications, approximately 70% of the 34 constraints are identical or semantically equivalent in all versions. Constraints like Non-negative Start Time (1), Non-negative Arrival Time (2), Start Time Gap (3), Flow Conservation (8), No Self-Loops (10), Resource Capacity Limit (12), Non-Delivery Node Load (15), and optimization constraints (33–34) are identical across all tests, ensuring consistent enforcement of timing, routing, and resource limits.

Minor syntactic variations, such as time(source,j) versus dist(source,j), Len(nodes) versus len(nodes), or Trips versus range(max_trips), appear frequently but carry no logical weight and do not affect solver behavior.

Despite being given the same natural language constraint, the LLM can translate them in different ways that are still logically valid. Some constraints differ significantly in structure and implications. For example, the Source Outgoing Route (4) constraint in Test 1 uses `Implies(trip_active, Sum == 1)`, which checks if there are any active trips, there must ensure there is an outgoing edge from the source while Tests 2 and 3 use `Sum == If(trip_active, 1, 0)` which states that the number of out going edges is 1 if that trip is active and 0 otherwise which makes it simple. Some generated constraints can also cover 2 constraints such as Test 1's generated constraint for Successor for Visited (24) and Source Successor (25).

Some constraints differ significantly in structure and implications. Delivery Time Separation (5) is different across tests with Test 1 enforcing separation between different drones using arrival time, Test 2 does the same but with delivery time and Test 3 missing it completely.

Test 3 also implemented constraints No.21 and No.22 into one generated constraint through an if condition. One constraints particularly for Active Trip Deliver Resource (28) is also missing in the third Test which is a crucial omission because, its' absence

could cause the solution to activate trips with zero payload across all nodes, thereby meeting flow and activation criteria while inflating trip counts, energy consumption, and optimization metrics.

A notable constraint which is the Sequential Trips (29) constraint has a drastic difference with Test 3 being the only one to include the time gap t_s into the constraint after a drone returns to the source before attempting to go for another trip. While the constraint in Test 1 and 2 may be complemented by the Trip Time Gap (20), its possible that it could get ignored and lead to infeasible schedules despite being satisfied in the model.

All three tests also generated one extra constraint, Drone Usage Indicator (35), which was not mentioned in the original natural language specifications.

```
ForAll([k], drone_used[k] == Or([trip_active[k,t] for t in
                range(max_trips)]))
```

This defines drone_used[k] as the logical OR of trip_active[k,t] across all trips—a derived indicator useful for fleet utilization analysis or secondary minimization goals. Its appearance in all tests could imply that the LLM deduced a useful constraint that would benefit solving the problem as the constraint could be useful for purposes such as minimizing energy.

7.1.2 Generated Z3 Constraints from Pseudocode

Table 5: Generated Z3 Constraints from Pseudocode (Test 1 vs. Test 2 vs. Test 3)

No.	Generated Z3 (Test 1)	Generated Z3 (Test 2)	Generated Z3 (Test 3)
1. Non-negative Start Time	`ForAll([k,t], start_time[k,t] >= 0)`	`ForAll([k,t], start_time[k,t] >= 0)`	`ForAll([k,t], start_time[k,t] >= 0)`
2. Non-negative Arrival Time	`ForAll([k,t,i], arrival_time[k,t,i] >= 0)`	`ForAll([k,t,i], arrival_time[k,t,i] >= 0)`	`ForAll([k,t,i], arrival_time[k,t,i] >= 0)`
3. Start Time Gap	`ForAll([k1,k2,t], Implies(k1 != k2, Abs(start_time[k1,t] - start_time[k2,t]) >= t_s))`	`ForAll([k1,k2,t], Implies(k1 != k2, Abs(start_time[k1,t] - start_time[k2,t]) >= {t_s}))`	`ForAll([k1,k2,t], Implies(k1 != k2, Abs(start_time[k1,t] - start_time[k2,t]) >= t_s))`
4. Source Outgoing Route	`ForAll([k,t], Implies(trip_active[k,t], Sum([If(X_delivery[k,t,source,j], 1, 0) for j in nodes]) == 1))`	`ForAll([k,t], Implies(trip_active[k,t], Sum([If(X_delivery[k,t,source,j], 1, 0) for j in nodes]) == 1))`	`ForAll([k,t], Implies(trip_active[k,t], Sum([If(X_delivery[k,t,source,j], 1, 0) for j in nodes]) == 1))`
5. Delivery Time Separation	`ForAll([k1,k2,t1,t2,i], Implies(And(Or(k1 != k2, t1 != t2), i in delivery_nodes, delivery_time[k1,t1,i] > 0, delivery_time[k2,t2,i] > 0), Abs(delivery_time[k1,t1,i] - delivery_time[k2,t2,i]) >= w))`	`ForAll([k1,k2,t1,t2,i], Implies(And(i in delivery_nodes, Or(k1 != k2, t1 != t2), delivery_time[k1,t1,i] > 0, delivery_time[k2,t2,i] > 0), Abs(delivery_time[k1,t1,i] - delivery_time[k2,t2,i]) >= {w}))`	`ForAll([k1,k2,t1,t2,i], Implies(And(i in delivery_nodes, Or(k1 != k2, t1 != t2), delivery_time[k1,t1,i] > 0, delivery_time[k2,t2,i] > 0), Abs(delivery_time[k1,t1,i] - delivery_time[k2,t2,i]) >= w))`
6. Source-to-Node Arrival	`ForAll([k,t,j], Implies(X_delivery[k,t,source,j], arrival_time[k,t,j] == start_time[k,t] + edges[(source,j)]['time']))`	`ForAll([k,t,j], Implies(X_delivery[k,t,source,j], arrival_time[k,t,j] == start_time[k,t] + t_source,j))`	`ForAll([k,t,j], Implies(X_delivery[k,t,source,j], arrival_time[k,t,j] == start_time[k,t] + edges[(source,j)]['time']))`

Continued on next page

Table 5: Generated Z3 Constraints from Pseudocode (Continued)

No.	Generated Z3 (Test 1)	Generated Z3 (Test 2)	Generated Z3 (Test 3)
7. Node-to-Node Arrival	`ForAll([k,t,i,j], Implies(And(i in delivery_nodes, X_delivery[k,t,i,j]), arrival_time[k,t,j] == arrival_time[k,t,i] + edges[(i,j)]['time'] + w))`	`ForAll([k,t,i,j], Implies(And(i in delivery_nodes, X_delivery[k,t,i,j]), arrival_time[k,t,j] == arrival_time[k,t,i] + t_i,j + {w}))`	`ForAll([k,t,i,j], Implies(And(i in delivery_nodes, X_delivery[k,t,i,j]), arrival_time[k,t,j] == arrival_time[k,t,i] + edges[(i,j)]['time'] + w))`
8. Flow Conservation	`ForAll([k,t,i], Sum([If(X_delivery[k,t,i,j], 1, 0) for j in nodes]) == Sum([If(X_delivery[k,t,j,i], 1, 0) for j in nodes]))`	`ForAll([k,t,i], Sum([If(X_delivery[k,t,i,j], 1, 0) for j in nodes]) == Sum([If(X_delivery[k,t,j,i], 1, 0) for j in nodes]))`	`ForAll([k,t,i], Sum([If(X_delivery[k,t,i,j], 1, 0) for j in nodes]) == Sum([If(X_delivery[k,t,j,i], 1, 0) for j in nodes]))`
9. Max Arrival Time	`ForAll([k,t,i], Implies(Sum([If(X_delivery[k,t,i,j], 1, 0) for j in nodes]) > 0, arrival_time[k,t,i] <= t_max))`	`ForAll([k,t,i], Implies(Sum([If(X_delivery[k,t,i,j], 1, 0) for j in nodes]) > 0, arrival_time[k,t,i] <= {t_max}))`	`ForAll([k,t,i], Implies(Sum([If(X_delivery[k,t,i,j], 1, 0) for j in nodes]) > 0, arrival_time[k,t,i] <= t_max))`
10. No Self-Loops	`ForAll([k,t,i], X_delivery[k,t,i,i] == 0)`	`ForAll([k,t,i], X_delivery[k,t,i,i] == 0)`	`ForAll([k,t,i], Not(X_delivery[k,t,i,i]))`
11. Resource Delivery Balance	`ForAll([i], Implies(i in delivery_nodes, Sum([resources_delivered[k,t,i] for k in drones, t in trips]) == resources_required[i]))`	`ForAll([i], Implies(i in delivery_nodes, Sum([resources_delivered[k,t,i] for k in drones for t in trips]) == resources_required[i]))`	`ForAll([i], Implies(i in delivery_nodes, Sum([resources_delivered[k,t,i] for k,t]) == resources_required[i]))`
12. Resource Capacity Limit	`ForAll([k,t], Sum([resources_delivered[k,t,i] for i in delivery_nodes]) <= drones[k]['capacity'])`	`ForAll([k,t], Sum([resources_delivered[k,t,i] for i in delivery_nodes]) <= drones[k]['capacity'])`	`ForAll([k,t], Sum([resources_delivered[k,t,i] for i in delivery_nodes]) <= drones[k]['capacity'])`
13. Source Load Balance	`ForAll([k], load[k,source] == Sum([resources_delivered[k,t,i] for t in trips, i in delivery_nodes]))`	`ForAll([k], load[k,source] == Sum([resources_delivered[k,t,i] for t in trips for i in delivery_nodes]))`	`ForAll([k], load[k,source] == Sum([resources_delivered[k,t,i] for t in trips for i in delivery_nodes]))`
14. Delivery Node Load	`ForAll([k,i], Implies(i in delivery_nodes, load[k,i] == -Sum([resources_delivered[k,t,i] for t in trips])))`	`ForAll([k,i], Implies(i in delivery_nodes, load[k,i] == -Sum([resources_delivered[k,t,i] for t in trips])))`	`ForAll([k,i], Implies(i in delivery_nodes, load[k,i] == -Sum([resources_delivered[k,t,i] for t in trips])))`
15. Non-Delivery Node Load	`ForAll([k,i], Implies(And(i != source, Not(i in delivery_nodes)), load[k,i] == 0))`	`ForAll([k,i], Implies(And(i != source, Not(i in delivery_nodes)), load[k,i] == 0))`	`ForAll([k,i], Implies(And(i != source, Not(i in delivery_nodes)), load[k,i] == 0))`
16. Source Resource Limit	`Sum([load[k,source] for k in drones]) <= resources_available[source]`	`Sum([load[k,source] for k in drones]) <= resources_available[source]`	`Sum([load[k,source] for k]) <= resources_available[source]`
17. Non-negative Delivery Amount	`ForAll([k,t,i], Implies(i in delivery_nodes, resources_delivered[k,t,i] >= 0))`	`ForAll([k,t,i], Implies(i in delivery_nodes, resources_delivered[k,t,i] >= 0))`	`ForAll([k,t,i], Implies(i in delivery_nodes, resources_delivered[k,t,i] >= 0))`
18. Trip Activation	`Implies(Sum([resources_required[i] for i in delivery_nodes]) > 0, Or([trip_active[k,t] for k in drones, t in trips]))`	`Implies(Sum([resources_required[i] for i in delivery_nodes]) > 0, Or([trip_active[k,t] for k in drones for t in trips]))`	`Implies(Sum([resources_required[i] for i in delivery_nodes]) > 0, Or([trip_active[k,t] for k,t]))`
19. No Delivery Implies Inactive Trip	`ForAll([k,t], Implies(Sum([resources_delivered[k,t,i] for i in delivery_nodes]) == 0, And(Not(trip_active[k,t]), Sum([If(X_delivery[k,t,i,j], 1, 0) for i in nodes, j in nodes]) == 0)))`	`ForAll([k,t], Implies(Sum([resources_delivered[k,t,i] for i in delivery_nodes]) == 0, And(Not(trip_active[k,t]), Sum([If(X_delivery[k,t,i,j], 1, 0) for i in nodes for j in nodes]) == 0)))`	`ForAll([k,t], Implies(Sum([resources_delivered[k,t,i] for i in delivery_nodes]) == 0, And(Not(trip_active[k,t]), Sum([If(X_delivery[k,t,i,j], 1, 0) for i,j]) == 0)))`

Continued on next page

Table 5: Generated Z3 Constraints from Pseudocode (Continued)

No.	Generated Z3 (Test 1)	Generated Z3 (Test 2)	Generated Z3 (Test 3)
20. Trip Time Gap	`ForAll([k,t], Implies(t < max_trips - 1, start_time[k,t+1] >= start_time[k,t] + t_s))`	`ForAll([k,t], Implies(t < max_trips - 1, start_time[k,t+1] >= start_time[k,t] + {t_s}))`	`ForAll([k,t], Implies(t < max_trips - 1, start_time[k,t+1] >= start_time[k,t] + t_s))`
21. Delivery Time Equals Arrival	`ForAll([k,t,i], Implies(And(i in delivery_nodes, resources_delivered[k,t,i] > 0), delivery_time[k,t,i] == arrival_time[k,t,i]))`	`ForAll([k,t,i], Implies(And(i in delivery_nodes, resources_delivered[k,t,i] > 0), delivery_time[k,t,i] == arrival_time[k,t,i]))`	`ForAll([k,t,i], Implies(And(i in delivery_nodes, resources_delivered[k,t,i] > 0), delivery_time[k,t,i] == arrival_time[k,t,i]))`
22. No Delivery, Zero Time	`ForAll([k,t,i], Implies(And(i in delivery_nodes, resources_delivered[k,t,i] == 0), delivery_time[k,t,i] == 0))`	`ForAll([k,t,i], Implies(And(i in delivery_nodes, resources_delivered[k,t,i] == 0), delivery_time[k,t,i] == 0))`	`ForAll([k,t,i], Implies(And(i in delivery_nodes, resources_delivered[k,t,i] == 0), delivery_time[k,t,i] == 0))`
23. Visited Node Equals Traversed Edges	`ForAll([k,t], Implies(trip_active[k,t], Sum([If(visited_nodes[k,t,i], 1, 0) for i in nodes]) == Sum([If(X_delivery[k,t,i,j], 1, 0) for i in nodes, j in nodes])))`	`ForAll([k,t], Implies(trip_active[k,t], Sum([If(visited_nodes[k,t,i], 1, 0) for i in nodes]) == Sum([If(X_delivery[k,t,i,j], 1, 0) for i in nodes for j in nodes])))`	`ForAll([k,t], Implies(trip_active[k,t], Sum([If(visited_nodes[k,t,i], 1, 0) for i in nodes]) == Sum([If(X_delivery[k,t,i,j], 1, 0) for i in nodes for j in nodes])))`
24. Successor for Visited Nodes	`ForAll([k,t,i], Implies(And(i != source, visited_nodes[k,t,i]), Or([And(X_delivery[k,t,i,j], successor[k,t,i] == node_indices[j]) for j in nodes if (i,j) in edges])))`	`ForAll([k,t,i], Implies(And(i != source, visited_nodes[k,t,i]), Or([And(X_delivery[k,t,i,j], successor[k,t,i] == node_indices[j]) for j in nodes if (i,j) in edges])))`	`ForAll([k,t,i], Implies(And(i != source, visited_nodes[k,t,i]), Or([And(X_delivery[k,t,i,j], successor[k,t,i] == node_indices[j]) for j in nodes if (i,j) in edges])))`
25. Source Successor	`ForAll([k,t], Implies(trip_active[k,t], Or([And(X_delivery[k,t,source,j], successor[k,t,source] == node_indices[j]) for j in nodes if (source,j) in edges])))`	`ForAll([k,t], Implies(trip_active[k,t], Or([And(X_delivery[k,t,source,j], successor[k,t,source] == node_indices[j]) for j in nodes if (source,j) in edges])))`	`ForAll([k,t], Implies(trip_active[k,t], Or([And(X_delivery[k,t,source,j], successor[k,t,source] == node_indices[j]) for j in nodes if (source,j) in edges])))`
26. No Successor for Unvisited Nodes	`ForAll([k,t,i], Implies(Not(visited_nodes[k,t,i]), successor[k,t,i] == -1))`	`ForAll([k,t,i], Implies(Not(visited_nodes[k,t,i]), successor[k,t,i] == -1))`	`ForAll([k,t,i], Implies(Not(visited_nodes[k,t,i]), successor[k,t,i] == -1))`
27. Active Trip Visit Delivery	`ForAll([k,t], Implies(trip_active[k,t], Sum([If(visited_nodes[k,t,i], 1, 0) for i in delivery_nodes]) >= 1))`	`ForAll([k,t], Implies(trip_active[k,t], Sum([If(visited_nodes[k,t,i], 1, 0) for i in delivery_nodes]) >= 1))`	`ForAll([k,t], Implies(trip_active[k,t], Sum([If(visited_nodes[k,t,i], 1, 0) for i in delivery_nodes]) >= 1))`
28. Active Trip Deliver Resource	`ForAll([k,t], Implies(trip_active[k,t], Sum([resources_delivered[k,t,i] for i in delivery_nodes]) > 0))`	`ForAll([k,t], Implies(trip_active[k,t], Sum([resources_delivered[k,t,i] for i in delivery_nodes]) > 0))`	`ForAll([k,t], Implies(trip_active[k,t], Sum([resources_delivered[k,t,i] for i in delivery_nodes]) > 0))`
29. Sequential Trips	`ForAll([k,t], Implies(And(trip_active[k,t], t < max_trips - 1), start_time[k,t+1] >= arrival_time[k,t,source]))`	`ForAll([k,t], Implies(And(trip_active[k,t], t < max_trips - 1), start_time[k,t+1] >= arrival_time[k,t,source]))`	`ForAll([k,t], Implies(And(trip_active[k,t], t < max_trips - 1), start_time[k,t+1] >= arrival_time[k,t,source]))`
30. Return to Source	`ForAll([k,t], Implies(trip_active[k,t], Sum([If(X_delivery[k,t,j,source], 1, 0) for j in nodes]) == 1))`	`ForAll([k,t], Implies(trip_active[k,t], Sum([If(X_delivery[k,t,j,source], 1, 0) for j in nodes]) == 1))`	`ForAll([k,t], Implies(trip_active[k,t], Sum([If(X_delivery[k,t,j,source], 1, 0) for j in nodes]) == 1))`
31. Energy Usage Calculation	`ForAll([k], energy_used[k] == Sum([If(X_delivery[k,t,i,j], edges[(i,j)]['distance'], 0) for t in trips, i in nodes, j in nodes]))`	`ForAll([k], energy_used[k] == Sum([If(X_delivery[k,t,i,j], edges[(i,j)]['distance'], 0) for t in trips for i in nodes for j in nodes if (i,j) in edges]))`	`ForAll([k], energy_used[k] == Sum([If(X_delivery[k,t,i,j], edges[(i,j)]['distance'], 0) for t,i,j]))`

Continued on next page

Table 5: Generated Z3 Constraints from Pseudocode (Continued)

No.	Generated Z3 (Test 1)	Generated Z3 (Test 2)	Generated Z3 (Test 3)
32. Energy Capacity Limit	`ForAll([k], energy_used[k] <= drones[k]['energy'])`	`ForAll([k], energy_used[k] <= drones[k]['energy'])`	`ForAll([k], energy_used[k] <= drones[k]['energy'])`
33. Minimize Total Energy	`Minimize(Sum([energy_used[k] for k in drones]))`	`Minimize(Sum([energy_used[k] for k in drones]))`	`Minimize(Sum([energy_used[k] for k]))`
34. Minimize Active Trips	`Minimize(Sum([If(trip_active[k,t], 1, 0) for k in drones, t in trips]))`	`Minimize(Sum([If(trip_active[k,t], 1, 0) for k in drones for t in trips]))`	`Minimize(Sum([If(trip_active[k,t], 1, 0) for k,t]))`

Analysis The table of generated Z3 constraints from pseudocode for the drone delivery scheduling problem reveals a high degree of similarity in formulation and structure across the three tests approximately 88%(30 of 34) of the constraints are identical or semantically equivalent Most constraints, such as Non-negative Start Time (1), Source Outgoing Route (4), Flow Conservation (8), No Self-Loops (10), Resource Delivery Balance (11) and optimization constraints (33–34), are identical across all Tests.

However, despite being prompted with the same structured pseudocode, some generated constraints differ greatly from each other. Few of the constraints contain minor syntactic differences, such as Delivery Time Separation (5), where Tests 1 and 3 uses the standard Z3 syntax (w, t_s) while Test 2 uses curly braces$(\{w\}, \{t_\})$, and Non-negative Arrival Time (2) and Max Arrival Time (9), where Test 3 contain different variable order ([i,k,t] vs. [k,t,i]).

This variation in orders could possibly lead to solver efficiency issues. The relationship between route selection and arrival timings becomes under-specified if these symbolic parameters are not consistently defined within the solver, possibly resulting in solutions that are either incorrect or not feasible.

The most notable difference is between tess for Energy Usage Calculation (31), where in Tests 1 and 3 define energy_used[k] as the sum of distances over all edges `Sum([If(X_delivery[k,t,i,j], edges[(i,j)]['distance'], 0 for t in trips, i in nodes, j in nodes]))`, potentially including invalid edges, which may lead to incorrect energy calculations if edges are undefined. In contrast, Test 2 includes a conditional check `if (i,j) in edges`, ensuring only valid edges contribute to the sum, improving accuracy but also increasing in computational complexity.

7.2 Generated Code Comparison

7.2.1 Generated Code from Using Natural Language Constraints

The code generated from natural language specifications to implement Z3 constraints for the drone delivery scheduling problem across three tests demonstrates varied approaches to translating the same constraints and implementing them into the code. Ignoring different syntax and structure, approximately 70% of the 34 constraints are implemented identically or with equivalent logic, ensuring consistent enforcement of constraints with several of the constraints are implemented differently throughout the tests. Below will discuss the different methods in how several constraints are implemented. Notably, Test 3 utilizes track function to store records of constraints for diagnostic and debugging

purposes but also add the constraint to Z3 solver, the optimization solver (opt) and the core solver (core_solver), enabling unsat core extraction.

```
def track(name, expr):
    # Record for unsat core
    b = Bool(f"cnstr::{name}")
    tracked_constraints.append((b, name))
    core_solver.assert_and_track(expr, b)
    opt.add(expr)
```

Non-negative Start Time (1)

- **Test 1:** Implemented by iterating each drone k and trip t, ensuring `start_time`$[k,t] \geq 0$ and bounded by `t_max`.

```
for k in K:
    for i in I:
        for t in T:
            start_time[(k, t)] = Int(f"start_time[{k},{t}]")
            opt.add(start_time[(k, t)] >= 0, start_time[(k, t)] <= t_max)

```

- **Test 2:** Identical logic, enforcing start_time$[k,t] \geq 0$ but not bounded by max_time.

```
for k in drone_names:
    for t in range(max_trips):
        start_time[(k, t)] = Int(f"start_time[{k},{t}]")
        o.add(start_time[(k, t)] >= 0)

```

- **Test 3:** Same as Test 2

Non-negative Arrival Time (2)

- **Test 1**: Enforces non-negativity with $t_{\max}$ bound: `opt.add(arrival_time[(k, t, i)] >= 0, arrival_time[(k, t, i)] <= t_max)`. Ensures valid timing.

```
for k in K:
    for t in T:
        for i in I:
            arrival_time[(k, t, i)] = Int(f"arrival_time[{k},{t},{i}]")
            opt.add(arrival_time[(k, t, i)] >= 0, arrival_time[(k, t, i)] <= t_max)

```

- **Test 2**: Same logic as Test 1 but not bounded by max time `t_max`.

```
for k in drone_names:
    for t in range(max_trips):
        for i in nodes:
            arrival_time[(k, t, i)] = Int(f"arrival_time[{k},{t},{i}]")
            o.add(arrival_time[(k, t, i)] >= 0)

```

- **Test 3**: Same Test 2

```
for k in K:
    for t in T:
        for i in I:
            track(f"arrival_nonneg[{k},{t},{i}]", arrival_time[(k, t, i)] >= 0)
            track(f"delivery_nonneg[{k},{t},{i}]", delivery_time[(k, t, i)] >= 0)

```

Start Time Gap (3)

- **Test 1**: Enforces separation using `Abs`: `opt.add(Abs(start_time[(k1, t)] - start_time[(k2, t)]) >= t_s)`. Iterates over distinct drone pairs for each trip.

```
for t in T:
    for i1, k1 in enumerate(K):
        for k2 in K[i1 + 1:]:
            opt.add(Abs(start_time[(k1, t)] - start_time[(k2, t)]) >= t_s)

```

- **Test 2**: Similar approach using `Abs`: `o.add(Implies(True, Abs(start_time[(k1, t)] - start_time[(k2, t)]) >= t_s))`. The `Implies(True, ...)` is redundant but aligns with parsing logic.

```
for idx1, k1 in enumerate(drone_names):
    for idx2, k2 in enumerate(drone_names):
        if k1 == k2:
            continue
        for t in range(max_trips):
            o.add(Implies(True, Abs(start_time[(k1, t)] - start_time[(k2, t)]) >= t_s))

```

- **Test 3**: Same logic as Test 1

Source Outgoing Route (4)

- **Test 1:** Enforces one outgoing edge for active trips: `opt.add(Implies(trip_active[(k,t)], out_from_source == 1))`, ensuring strict path continuity from the source node.

```

def B2I(b): return If(b, 1, 0)
for k in K:
        for t in T:
            out_from_source = Sum([B2I(X[(k, t, source, j)]) for j in outgoing.get(source, [])])
            into_source = Sum([B2I(X[(k, t, i, source)]) for i in incoming.get(source, [])])
            opt.add(Implies(trip_active[(k, t)], out_from_source == 1))
```

- **Test 2:** Uses helper function `sum_edges_from_source(k,t)` and equates to `If(trip_active[k,t], 1, 0)`.
`o.add(sum_edges_from_source(k,t) == If(trip_active[(k,t)], 1, 0))`.
This simplifies logic but if `trip_active[k,t] = False`,
Implies Inactive Trip (19) and Flow Conservation (8) constraints prevent edge usage.

```
def sum_edges_from_source(k, t):
    return Sum([If(X_delivery[(k, t, source, j)], 1, 0) for j in nodes if (source, j) in directed_edges])

for k in drone_names:
    for t in range(max_trips):
        o.add(sum_edges_from_source(k, t) == If(trip_active[(k, t)], 1, 0))
```

- **Test 3:** Counts outgoing edges using `sum_bools(outs)` and tracked using `track(f"src_out_equal`
`sum_bools(outs) ==`
`If(trip_active[(k,t)], IntVal(1), IntVal(0)))`. Functionally similar to Test 2.

```
def sum_bools(bools):
            return Sum([If(b, IntVal(1), IntVal(0)) for b in bools]) if bools else IntVal(0)
for k in K:
    for t in T:
        outs = [X_delivery[(k, t, source, j)] for j in J if j != source and (source, j) in edges]
        track(f"src_out_equals_active[{k},{t}]",
              sum_bools(outs) == If(trip_active[(k, t)], IntVal(1), IntVal(0)))
        track(f"src_in_equals_active[{k},{t}]",
              sum_bools(ins) == If(trip_active[(k, t)], IntVal(1), IntVal(0)))
```

Delivery Time Separation (5)

- **Test 1:** Iterates over drone pairs, enforcing a minimum time separation `w` between arrival times at delivery nodes for different drone-trip pairs`Abs(arrival_time[k1,t1,i] - arrival_time[k2,t2,i]) >= w`

```
for i in D:
    for k1 in K:
        for t1 in T:
            for k2 in K:
                for t2 in T:
                    if (k1, t1) < (k2, t2):
                        opt.add(Implies(And(resources_delivered[(k1, t1, i)] > 0,
                                            resources_delivered[(k2, t2, i)] > 0),
                                        Abs(arrival_time[(k1, t1, i)] - arrival_time[(k2, t2, i)]) >= w))
```

- **Test 2:** Similar to Test 1 but uses delivery_times, instead which aligns better with intent but introducing conditional complexity.

```
for k1 in drone_names:
    for k2 in drone_names:
        if k1 == k2:
            continue
        for t1 in range(max_trips):
            for t2 in range(max_trips):
                for i in delivery_nodes:
                    ri1 = resources_delivered[(k1, t1, i)]
                    ri2 = resources_delivered[(k2, t2, i)]
                    o.add(Implies(And(ri1 > 0, ri2 > 0),
                                  Abs(delivery_time[(k1, t1, i)] - delivery_time[(k2, t2, i)]) >= w))
```

- **Test 3:** Missing in Implementation

Source to Node Arrival (6)

- **Test 1:** Implemented by setting `arrival_time[(k,t,j)]` = `start_time[(k,t)] + time_of(source,j)` for edges from the source when `X[(k,t,source,j)]` is true.

```
for k in K:
    for t in T:
        for i in I:
            for j in outgoing.get(i, []):
                base = If(i == source,
                          start_time[(k, t)],
                          arrival_time[(k, t, i)] + (w if i in delivery_nodes else 0))
                opt.add(Implies(X[(k, t, i, j)],
                                arrival_time[(k, t, j)] == base + time_of(i, j)))
```

- **Test 2**:

```
for k in drone_names:
    for t in range(max_trips):
        for j in nodes:
            if (source, j) in directed_edges:
                o.add(Implies(X_delivery[(k, t, source, j)],
                              arrival_time[(k, t, j)] == start_time[(k, t)] + dist(source, j)))
```

- **Test 3**: Same as Test 2

Node-to-Node Arrival (7)

- **Test 1**: Iterates through `base = arrival_time[(k,t,i)] + (w if i in delivery_nodes else 0)`, assuming service time for all delivery nodes, which may over-constrain if no resources are delivered.

```
for k in K:
    for t in T:
        for i in I:
            for j in outgoing.get(i, []):
                base = If(i == source, start_time[(k,t)], arrival_time[(k,t,i)] + (w if i in delivery_nodes
 else 0))
                opt.add(Implies(X[(k,t,i,j)], arrival_time[(k,t,j)] == base + time_of(i,j)))

```

- **Test 2**: Adds a conditions to add service time on delivery with texttttIf(resources_delivered[k,t,i] ¿ 0, w, 0), ensuring precise timing.

```
for k in drone_names:
    for t in range(max_trips):
        for i in nodes:
            for j in nodes:
                if (i, j) in directed_edges and i != source:
                    ri = resources_delivered[(k, t, i)] if (k, t, i) in resources_delivered else Int(f"
 _zero_{k}_{t}_{i}")
                    if (k, t, i) not in resources_delivered:
                        o.add(ri == 0)
                        resources_delivered[(k, t, i)] = ri
                    o.add(Implies(X_delivery[(k, t, i, j)],
                                  arrival_time[(k, t, j)] == arrival_time[(k, t, i)] + dist(i, j) + If(ri >
 0, w, 0)))

```

- **Test 3**: Applies service time `w` for all edges from $i \neq source$, with `arrival_time[(k,t,j)] == arrival_time[(k,t,i)] + IntVal(w + time_map[(i,j)])`. This over-constrains by including `w` even when no resources are delivered.

```
for k in K:
    for t in T:
        for i in I:
            for j in J:
                if i != source and j != source and (i,j) in edges:
                    track(f"time_step[{k},{t},{i},{j}]", Implies(X_delivery[(k,t,i,j)], arrival_time[(k,t,j)
 ] == arrival_time[(k,t,i)] + IntVal(w + time_map[(i,j)])))
                if i != source and j == source and (i,j) in edges:
                    track(f"time_to_src[{k},{t},{i},{j}]", Implies(X_delivery[(k,t,i,j)], arrival_time[(k,t,
 source)] == arrival_time[(k,t,i)] + IntVal(w + time_map[(i,j)])))

```

Flow Conservation (8)

- **Test 1:** Implemented using direct summations of outgoing edges: `(out_sum = Sum([B2I(X[(k,t,i,j)]) for j in outgoing.get(i, [])])` and incoming edges: `in_sum = Sum([B2I(X[(k,t,j,i)]) for j in incoming.get(i, [])]))`, checking with Boolean to integer function `(B2I)` to count outgoing and incoming edge per active trip `opt.add(Implies(trip_active[(k,t)], out_from_source == 1))`. This generates constraints for all nodes, relying on separate constraints to nullify invalid edges.

```
for k in K:
    for t in T:
        for i in I:
            out_sum = Sum([B2I(X[(k, t, i, j)]) for j in outgoing.get(i, [])])
            in_sum = Sum([B2I(X[(k, t, j, i)]) for j in incoming.get(i, [])])
            opt.add(visited[(k, t, i)] == Or(out_sum == 1, in_sum == 1))
            # Flow conservation: in == out
            opt.add(out_sum == in_sum)
            # At most one in and one out per node per trip
            opt.add(out_sum <= 1, in_sum <= 1)
```

- **Test 2:** Utilizes helper functions `sum_out(k,t,i)` and `sum_in(k,t,i)` which compute edge counts only for valid edges `(if (i,j) in directed_edges)`, adding `o.add(sum_in(k,t,i) == sum_out(k,t,i))`. This reduces the application of constraints on all edges, prioritizing only valid ones.

```
def sum_out(k, t, i):
    return Sum([If(X_delivery[(k, t, i, j)], 1, 0) for j in nodes if (i, j) in directed_edges])
def sum_in(k, t, i):
    return Sum([If(X_delivery[(k, t, j, i)], 1, 0) for j in nodes if (j, i) in directed_edges])

for k in drone_names:
    for t in range(max_trips):
        for i in nodes:
            o.add(sum_in(k, t, i) == sum_out(k, t, i))
```

- **Test 3:** Uses a `sum_bools` function to count incoming and outgoing edges `sum_bools(in_edges) == sum_bools(out_edges)`, with explicit edge checks `(if (i,j) in edges)` similar to Test 2.

```
def sum_bools(bools):
    return Sum([If(b, IntVal(1), IntVal(0)) for b in bools]) if bools else IntVal(0)
for k in K:
    for t in T:
        for i in I:
            out_edges = [X_delivery[(k, t, i, j)] for j in J if (i, j) in edges]
            in_edges = [X_delivery[(k, t, j, i)] for j in J if (j, i) in edges]
            track(f"flow_conserve[{k},{t},{i}]",
                  sum_bools(in_edges) == sum_bools(out_edges))
```

Max Arrival Time(9)

- **Test 1**:

```
for k in K:
    for t in T:
        for i in I:
            opt.add(Implies(visited[(k, t, i)], arrival_time[(k, t, i)] <= t_max))
```

- **Test 2**: Same as Test 1
- **Test 3**: Same as Test 1

No Self-Loops (10)

- **Test 1**:

```
for k in K:
    for t in T:
        for i in I:
            for j in J:
                # Forbid self-loops
                if i == j:
                    opt.add(Not(X[(k, t, i, j)]))
```

- **Test 2**: Same logic as Test 1

```
for k in drone_names:
    for t in range(max_trips):
        for i in nodes:
            # no self-loop
            o.add(Not(X_delivery[(k, t, i, i)]))
```

- **Test 3**: Same as Test 1

Resource Delivery Balance (11)

- **Test 1**:

```
for i in D:
    total_to_i = Sum([resources_delivered[(k, t, i)] for k in K for t in T])
    opt.add(total_to_i == int(resources_required.get(i, 0)))
```

- **Test 2**:

```
for i in delivery_nodes:
    terms = []
    for k in drone_names:
        for t in range(max_trips):
            terms.append(resources_delivered[(k, t, i)])
    o.add(Sum(terms) == int(resources_required.get(i, 0)))
```

- **Test 3**: Same as Test 1

Resource Capacity Limit (12)

- **Test 1**:

```
for k in K:
    cap_k = int(drones[k].get('capacity', 0))
    for t in T:
        # sum delivered in this trip <= capacity
        opt.add(sum_trip_deliv <= cap_k)
```

- **Test 2**:

```
 for k in drone_names:
    cap = int(drones_info[k].get('capacity', 0))
    for t in range(max_trips):
        o.add(Sum([resources_delivered.get((k, t, i), Int(f"_zero2_{k}_{t}_{i}")) for i in delivery_nodes])
     <= cap)
```

- **Test 3**:

```
for k in K:
    for t in T:
        sum_trip_deliv = Sum([resources_delivered[(k, t, i)] for i in delivery_nodes]) if delivery_nodes
     else IntVal(0)
        track(f"cap_trip[{k},{t}]", sum_trip_deliv <= IntVal(capacity[k]))
```

Source Load Balance (13), Delivery Node Load (14),Non-Deliver Node Load (15) and Source Resource Limit (16)

- **Test 1**:

```
# Load constraints
for k in K:
    opt.add(load[(k, source)] == Sum([resources_delivered[(k, t, i)] for t in T for i in D]))
    for i in I:
        if i in D:
            opt.add(load[(k, i)] == -Sum([resources_delivered[(k, t, i)] for t in T]))
        elif i != source:
            opt.add(load[(k, i)] == 0)
opt.add(Sum([load[(k, source)] for k in K]) <= int(total_available_at_source))

```

- **Test 2**:

```
elif s_stripped == "Sum([load[k,source] for k in drones]) == Sum([resources_delivered[k,t,i] for k in drones
      for t in range(max_trips) for i in delivery_nodes])":
                left = Sum([load[(k, source)] for k in drone_names])
                right_terms = []
                for k in drone_names:
                    for t in range(max_trips):
                        for i in delivery_nodes:
                            right_terms.append(resources_delivered[(k, t, i)])
                o.add(left == Sum(right_terms))

            elif s_stripped == "ForAll([k,i], Implies(i in delivery_nodes, load[k,i] == - Sum([
     resources_delivered[k,t,i] for t in range(max_trips)])))":
                for k in drone_names:
                    for i in delivery_nodes:
                        o.add(load[(k, i)] == -Sum([resources_delivered[(k, t, i)] for t in range(max_trips)
     ]))

            elif s_stripped == "ForAll([k,i], Implies(And(i != source, Not(i in delivery_nodes)), load[k,i]
     == 0))":
```

```
                for k in drone_names:
                    for i in nodes:
                        if i != source and i not in delivery_nodes:
                            o.add(load[(k, i)] == 0)

            elif s_stripped == "Sum([load[k,source] for k in drones]) <= resources_available[source]":
                avail = int(resources_at.get(source, 0))
                o.add(Sum([load[(k, source)] for k in drone_names]) <= avail)

```

- **Test 3**:

```
total_required = sum([int(resources_required.get(i, 0)) for i in delivery_nodes])
track("load_source_total", Sum([load[(k, source)] for k in K]) == IntVal(total_required))
# - Load at source bounded by available resources
track("load_source_available", Sum([load[(k, source)] for k in K]) <= IntVal(resources_available.get(source,
      0)))
# - Loads at delivery nodes equal negative delivered by that drone
for k in K:
    for i in I:
        if i in delivery_nodes:
            sum_deliv_k_i = Sum([resources_delivered[(k, t, i)] for t in T])
            track(f"load_deliv_node[{k},{i}]", load[(k, i)] == -sum_deliv_k_i)
        elif i != source:
            track(f"load_non_deliv_non_source[{k},{i}]", load[(k, i)] == IntVal(0))

```

Non-negative Delivery Amount (17)

- **Test 1**

```
for k in K:
    for t in T:
        for i in I:
            # No delivery to non-delivery nodes
            if i not in D:
                opt.add(resources_delivered[(k, t, i)] == 0)
            else:
                # 0 <= delivered <= min(capacity, remaining required upper bound)
                req_i = int(resources_required.get(i, 0))
                opt.add(resources_delivered[(k, t, i)] >= 0)
```

- **Test 2**: Bounded by capacity

- **Test 3**: Bounded by capacity

```
for k in K:
    for t in T:
        # Resources delivered non-negative and bounded by capacity
        for i in delivery_nodes:
            track(f"res_deliv_nonneg[{k},{t},{i}]", resources_delivered[(k, t, i)] >= 0)
            track(f"res_deliv_cap[{k},{t},{i}]",
                  resources_delivered[(k, t, i)] <= capacity[k])

```

Trip Activation (18) and No Delivery Implies Inactive Trip(19)

- **Test 1**:

```
for k in K:
    for t in T:
        # sum delivered in this trip <= capacity
        sum_trip_deliv = Sum([resources_delivered.setdefault((k, t, i), Int(f"resources_delivered[{k},{t},{i
     }]")) for i in D])

        opt.add(Implies(sum_trip_deliv > 0, trip_active[(k, t)]))
        opt.add(Implies(sum_trip_deliv == 0, Not(trip_active[(k, t)])))

```

- **Test 2**: Same as Test 1

- **Test 3**: Same as Test 1

Time Trip Gap (20)

- **Test 1**:

```
for k in K:
    for t in T:
        if t > 0:
            opt.add(start_time[(k, t)] >= start_time[(k, t - 1)] + t_s)
            opt.add(start_time[(k, t)] >= arrival_time[(k, t - 1, source)])
```

- **Test 2**:

```
for k in drone_names:
    for t in range(1, max_trips):
        o.add(start_time[(k, t)] >= start_time[(k, t - 1)] + t_s)
```

- **Test 3**:

```
for k in K:
    for t in T:
        if t > 0:
            track(f"start_gap_own[{k},{t}]",
                  start_time[(k, t)] >= start_time[(k, t - 1)] + IntVal(t_s))
            # Also after previous trip's arrival at source
            track(f"start_after_prev_end[{k},{t}]",
                  start_time[(k, t)] >= arrival_time[(k, t - 1, source)] + IntVal(t_s))
```

Delivery Time Equals Arrival (21) and No Delivery, Zero Time (22):

- **Test 1**: Implemented the delivery–arrival relationship only when a delivery occurs. If resources are delivered at node i, the delivery time matches the arrival time: `opt.add(Implies(resources_delivered[(k,t,i)] > 0, delivery_time[(k,t,i)] == arrival_time[(k,t,i)]))`,
  otherwise, delivery time is forced to zero:
  `opt.add(Implies(resources_delivered[(k,t,i)] == 0, delivery_time[(k,t,i)] == 0))`
  However, this approach does not explicitly define delivery times at non-delivery nodes, leaving some variables unconstrained.

```
for k in K:
    for t in T:
        for i in I:
            opt.add(Implies(resources_delivered[(k,t,i)] > 0, delivery_time[(k,t,i)] == arrival_time[(k,t,i)
    ]))
            opt.add(Implies(resources_delivered[(k,t,i)] == 0, delivery_time[(k,t,i)] == 0))

```

- **Test 2**: Similar, but extends Test 1 by explicitly setting `delivery_time[(k,t,i)] = 0` for all non-delivery nodes. This eliminates unconstrained delivery-time variables, improving model consistency

```
for k in drone_names:
    for t in range(max_trips):
        for i in delivery_nodes:
            o.add(Implies(resources_delivered[(k,t,i)] > 0, delivery_time[(k,t,i)] == arrival_time[(k,t,i)])
    )
        for i in nodes:
            if i in delivery_nodes:
                o.add(Implies(resources_delivered[(k,t,i)] == 0, delivery_time[(k,t,i)] == 0))
            else:
                o.add(delivery_time[(k,t,i)] == 0)

```

- **Test 3**: Combines both constraints into a single If statement: `If(resources_delivered[(k,t,i)] > 0, delivery_time[(k,t,i)] == arrival_time[(k,t,i)], delivery_time[(k,t,i)] == 0)`. Non-delivery nodes are handled separately.

```
for k in K:
    for t in T:
        for i in I:
            if i in delivery_nodes:
                track(f"deliv_time_def[{k},{t},{i}]", If(resources_delivered[(k,t,i)] > 0, delivery_time[(k,
 t,i)] == arrival_time[(k,t,i)], delivery_time[(k,t,i)] == IntVal(0)))
            else:
                track(f"non_delivery_zero[{k},{t},{i}]", delivery_time[(k,t,i)] == IntVal(0))

```

Visited Node Equals Traversed Edges (23)

- **Test 1**:

```
for k in K:
    for t in T:
        for i in I:
            opt.add(Implies(resources_delivered[(k,t,i)] > 0, delivery_time[(k,t,i)] == arrival_time[(k,t,i)
 ]))
            opt.add(Implies(resources_delivered[(k,t,i)] == 0, delivery_time[(k,t,i)] == 0))

```

- **Test 2**: This constraint got skipped. This will impact the solver greatly as it itll

```
elif s_stripped == "ForAll([k,t], Implies(trip_active[k,t], Sum([If(visited_nodes[k,t,i], 1, 0) for i in
 nodes]) == Sum([If(X_delivery[k,t,i,j], 1, 0) for i in nodes for j in nodes]))":
                # This pattern is inconsistent in general (edges count vs visited nodes).
                # We will relax by skipping and warn.
                warnings.append("Warning: Skipped inconsistent visited_nodes == edges_used equality
 constraint.")

```

- **Test 3**:

```
for k in K:
    for t in T:
        opt.add(Implies(trip_active[(k, t)],
                        Sum([B2I(visited[(k, t, i)]) for i in I]) ==
                        Sum([B2I(X[(k, t, i, j)]) for i in I for j in outgoing.get(i, [])])))

```

Successor for Visited Nodes (24)

- **Test 1**: Implemented by Also used for Source Successor Constraint (25)

```
for k in K:
    for t in T:
        for i in I:
            outs = outgoing.get(i, [])
            if outs:
                # Make a disjunction that for each j, if the edge is used, successor == node_indices[j]
                succ_cases = []
                for j in outs:
                    succ_cases.append(And(X[(k, t, i, j)], successor[(k, t, i)] == node_indices[j]))
                opt.add(Implies(visited[(k, t, i)], Or(succ_cases)))
            else:
                # If no outgoing edges available, cannot be visited
                opt.add(Not(visited[(k, t, i)]))

```

- **Test 2**: Correctly uses an existential quantifier, enforcing `o.add(Implies(visited_nodes_var[(k,t,i)], Or(disj)))` where `disj` includes `And(successor[(k,t,i)] == j_idx, X_delivery[(k,t,i,j_node)])` for valid edges.

```
for k in drone_names:
    for t in range(max_trips):
        for i in nodes:
            disj = []
            for j_idx in range(len(nodes)):
                j_node = idx_to_node[j_idx]
                # Only meaningful if edge exists
                if (i, j_node) in directed_edges:
```

```
                    disj.append(And(successor[(k, t, i)] == j_idx, X_delivery[(k, t, i, j_node)]))
            if disj:
                o.add(Implies(visited_nodes_var[(k, t, i)], Or(disj)))
            else:
                # If no outgoing edges exist from i, visited_nodes must be false
                o.add(Not(visited_nodes_var[(k, t, i)]))
```

- **Test 3**: Explicitly handles the source, with `track(f"succ_src_active[k,t]", Implies(trip_active[(k,t)], And(successor[(k,t,source)] >= 0, ..., Or(*choices_src` where `choices_src` matches edges to successor indices.

```
for k in K:
    for t in T:
        choices_src = []
        for j in J:
            if (source,j) in edges:
                choices_src.append(And(successor[(k,t,source)] == IntVal(node_indices[j]), X_delivery[(k,t,
     source,j)]))
        if choices_src:
            track(f"succ_src_active[{k},{t}]", Implies(trip_active[(k,t)], And(successor[(k,t,source)] >= 0,
      successor[(k,t,source)] < IntVal(len(nodes)), Or(*choices_src))))

```

Source Successor (25)

- **Test 1:** Implemented in (24).
- **Test 2:**

```
for k in drone_names:
    for t in range(max_trips):
        disj = []
        for j_idx in range(len(nodes)):
            j_node = idx_to_node[j_idx]
            if (source, j_node) in directed_edges:
                disj.append(And(X_delivery[(k, t, source, j_node)], successor[(k, t, source)] == j_idx))
        if disj:
            o.add(Implies(trip_active[(k, t)], Or(disj)))
        else:
            # No outgoing from source -> cannot have active trip
            o.add(Not(trip_active[(k, t)]))
```

- **Test 3:**

No Successor for Unvisited Nodes (26)

- **Test 1:**

```
for k in K:
    for t in T:
        for i in I:
            opt.add(Implies(Not(visited[(k, t, i)]),
                            successor[(k, t, i)] == -1))
```

- **Test 2:** Same as Test 1
- **Test 3:** Same as Test 1

Active Trip Visit Delivery (27)

- **Test 1:** It implied `opt.add(Implies(trip_active[(k,t)]`

```
for k in K:
    for t in T:
        opt.add(Implies(trip_active[(k, t)],
            Sum([B2I(visited[(k, t, i)]) for i in D]) >= 1))
```

- **Test 2:** Same as Test 1
- **Test 3:** Also used for Active Trip Deliver Resource (28)

```
# Active trip must visit at least one delivery node
for k in K:
    for t in T:
        track(f"active_visits_delivery[{k},{t}]",
              Implies(trip_active[(k, t)],
                      Sum([If(visited_nodes_var[(k, t, i)], IntVal(1), IntVal(0)) for i in delivery_nodes])
     >= 1 if delivery_nodes else IntVal(0)))
```

Active Trip Deliver Resource (28)

- **Test 1:** It implies `opt.add(Implies(trip_active[(k,t)`, Sum([resources_delivered[(k,t,i)] for i in D]) ¿= 1)).

```
for k in K:
    for t in T:
        opt.add(Implies(trip_active[(k, t)], Sum([resources_delivered[(k, t, i)] for i in D]) >= 1))
```

- **Test 2:** Same as Test 1
- **Test 3:** Despite the generated constraint being missing, it was implemented. in **Active Trip Visit Delivery: (27)**

Sequential Trips (29)

- **Test 1:** Ensures start time `start_time[(k,t)]` is `>= arrival_time[(k,t-1,source)] + t_s`.

```
# Trip t must start after previous trip's start + t_s and after previous trip end at source
    if t > 0:
        opt.add(start_time[(k, t)] >= start_time[(k, t - 1)] + t_s)
        opt.add(start_time[(k, t)] >= arrival_time[(k, t - 1, source)])
```

- **Test 2:** Same as Test 1.
- **Test 3:** Same as Test 1.

Return to Source (30)

- **Test 1:** Enforces one incoming edge to the source for active trips `opt.add(Implies(trip_active[(k,t)], into_source == 1))`, where `into_source = Sum([B2I(X[(k,t,i,source)]) for i in incoming.get(source, [])])`, ensuring drones return to c.

```
for k in K:
    for t in T:
        out_from_source = Sum([B2I(X[(k, t, source, j)]) for j in outgoing.get(source, [])])
        into_source = Sum([B2I(X[(k, t, i, source)]) for i in incoming.get(source, [])])
        # Active trip => exactly one departure from and one return to source
        opt.add(Implies(trip_active[(k, t)], out_from_source == 1))
        opt.add(Implies(trip_active[(k, t)], into_source == 1))
```

- **Test 2:** Uses helper `sum_edges_to_source(k,t)` and equates it to `If(trip_active[k,t], 1, 0)`: `o.add(Implies(trip_active[(k,t)], sum_edges_to_source(k,t) == 1))`.

```
def sum_edges_to_source(k, t):
    return Sum([If(X_delivery[(k, t, i, source)], 1, 0) for i in nodes if (i, source) in directed_edges])

elif s_stripped == "ForAll([k,t], Implies(trip_active[k,t], Sum([If(X_delivery[k,t,i,source], 1, 0) for i in
      nodes]) == 1))":
    for k in drone_names:
        for t in range(max_trips):
            o.add(Implies(trip_active[(k, t)], sum_edges_to_source(k, t) == 1))
```

- **Test 3:** Similar to Test 2, uses `sum_bools(ins)`: `track(f"src_in_equals_active[k,t]", sum_bools(ins) == If(trip_active[(k,t)], IntVal(1), IntVal(0)))`.

```
for k in K:
    for t in T:
        outs = [X_delivery[(k, t, source, j)] for j in J if j != source and (source, j) in edges]
        ins = [X_delivery[(k, t, i, source)] for i in I if i != source and (i, source) in edges]
        track(f"src_out_equals_active[{k},{t}]",
              sum_bools(outs) == If(trip_active[(k, t)], IntVal(1), IntVal(0)))
        track(f"src_in_equals_active[{k},{t}]",
              sum_bools(ins) == If(trip_active[(k, t)], IntVal(1), IntVal(0)))
```

Energy Usage Calculation (31)

- **Test 1**:

```
# Energy usage
for k in K:
    energy_k = int(drones[k].get('energy', 0))
    sum_energy = []
    for t in T:
        for i in I:
            for j in outgoing.get(i, []):
                sum_energy.append(If(X[(k, t, i, j)], dist_of(i, j), 0))
    opt.add(energy_used[k] == Sum(sum_energy) if sum_energy else energy_used[k] == 0)

```

- **Test 2**:

```
for k in drone_names:
    o.add(energy_used[k] == Sum([If(X_delivery[(k, t, i, j)], dist(i, j), 0) for t in range(max_trips) for i
      in nodes for j in nodes if (i, j) in directed_edges]))

```

- **Test 3**:

```
for k in K:
    sum_dist = []
    for t in T:
        for (i, j) in nodes_pairs():
            sum_dist.append(If(X_delivery[(k, t, i, j)], IntVal(dist_map[(i, j)]), IntVal(0)))
    track(f"energy_def[{k}]", energy_used[k] == (Sum(sum_dist) if sum_dist else IntVal(0)))


```

Energy Capacity Limit (32)

- **Test 1**:

```
for k in K:
    energy_k = int(drones[k].get('energy', 0))
    opt.add(energy_used[k] <= energy_k)
```

- **Test 2**: Same as Test 1

- **Test 3**:

```
energy_cap = {k: int(drones[k].get('energy', 0)) for k in K}
for k in K:
    track(f"energy_cap[{k}]", energy_used[k] <= energy_cap[k])
```

Minimize Total Energy (33) and Minimize Active Trips (34)

- **Test 1**:

```
# Objectives
total_energy = Sum([energy_used[k] for k in K]) if K else Int("zero_energy")
total_trips = Sum([B2I(trip_active[(k, t)]) for k in K for t in T]) if K else Int("zero_trips")
opt.minimize(total_energy)
opt.minimize(total_trips)

```

- **Test 2**: Same as Test 1
- **Test 3**: Same as Test 1

Analysis Despite some constraints being missing or even combined, it allowed the LLM to generate codes that will help cover the missing constraints. Example is Test 2 includes the use of nested functions such as `sum_edges_from_source`, `sum_edges_to_source` and `sum_edges_trip` while Test 1 uses direct summations.

7.2.2 Generated code from Pseudo-code

Non-negative Start Time (1)

- **Test 1:** Implemented by iterating each drone k and trip t, ensuring start_time$[k, t] \geq 0$.

```
for k in drone_keys:
    for t in trips:
        opt.add(start_time[(k, t)] >= 0)

```

- **Test 2:** Identical logic, enforcing start_time$[k, t] \geq 0$ but bounded by max_time.

```
for k in drone_ids:
    for t in trips:
        # start time bounds
        opt.add(start_time[k][t] >= 0, start_time[k][t] <= t_max)

```

- **Test 3:** Same as Test 1

Non-negative Arrival Time (2)

- **Test 1**: Enforces non-negativity: `opt.add(arrival_time[(k, t, i)] >= 0, arrival_time[(k, t, i)])` together with `opt.add(delivery_time[(k, t, i)] >= 0, delivery_time[(k, t, i)])` Ensures valid timing.

```
for k in drone_keys:
    for t in trips:
        for i in nodes:
            opt.add(arrival_time[(k, t, i)] >= 0)
            opt.add(delivery_time[(k, t, i)] >= 0)

```

- **Test 2**: Same logic as Test 1 but not bounded by max time `t_max`.

```
for k in drone_keys:
    for t in trips:
        for i in nodes:
            opt.add(arrival_time[(k, t, i)] >= 0, arrival_time[(k, t, i)] <= t_max)
            opt.add(delivery_time[(k, t, i)] >= 0, delivery_time[(k, t, i)] <= t_max)

```

- **Test 3**: Same logic as Test 1

Start Time Gap (3)

- **Test 1**: Enforces separation using `Abs`: `opt.add(Abs(start_time[(k1, t)] - start_time[(k2, t)]) >= t_s)`. Iterates over distinct drone pairs for each trip.

```
# 3. start time separation across drones at same t
for t in trips:
    for idx1 in range(len(drone_keys)):
        for idx2 in range(len(drone_keys)):
            if idx1 == idx2:
                continue
            k1 = drone_keys[idx1]
            k2 = drone_keys[idx2]
            opt.add(zabs(start_time[(k1, t)] - start_time[(k2, t)]) >= t_s)

```

- **Test 2**: Similar approach using `Abs`: `o.add(Implies(True, Abs(start_time[(k1, t)] - start_time[(k2, t)]) >= t_s))`. The `Implies(True, ...)` is redundant but aligns with parsing logic.

```
for t in trips:
    for idx1, k1 in enumerate(drone_ids):
        for idx2, k2 in enumerate(drone_ids):
            if idx1 < idx2:
                # Replace Abs(a-b) >= t_s with equivalent Or to avoid potential solver issues
                opt.add(Or(start_time[k1][t] - start_time[k2][t] >= t_s,
                       start_time[k2][t] - start_time[k1][t] >= t_s))

```

- **Test 3**: Same logic as Test 1 but without checking if trips are active

```
for t in trips_range:
    for i1, k1 in enumerate(drone_ids):
        for k2 in drone_ids[i1 + 1:]:
            hard.add(Abs(start_time[(k1, t)] - start_time[(k2, t)]) >= t_s)
```

Source Outgoing Route (4)

- **Test 1:** Implemented using `sum_bools([X_delivery[(k,t,source,j)] for j in nodes])`, enforcing one outgoing edge from source node for active trips.

```
source = initial_node
for k in drone_keys:
    for t in trips:
        out_from_source = sum_bools([X_delivery[(k, t, source, j)] for j in nodes])
        opt.add(Implies(trip_active[(k, t)], out_from_source == 1))
```

- **Test 2:** Identical logic, implemented via `sum_out(k,t,initial_node)`, summing only valid edges, reducing unnecessary computations.

```
# Active trip implies exactly one out from source and exactly one into source
for k in drone_ids:
    for t in trips:
        opt.add(Implies(trip_active[k][t], sum_out(k, t, initial_node) == 1))
```

- **Test 3:** Same logic, using but also checks if the edge exist

```
def edge_exists(i, j):
            return (i, j) in edges_set
for k in drone_ids:
    for t in trips_range:
        out_from_source = zsum_bools([X_delivery[(k, t, source, j)] for j in nodes if edge_exists(source, j)
     ])
        hard.add(Implies(trip_active[(k, t)], out_from_source == 1))
```

Delivery Time Separation (5)

- **Test 1:** Implements using function `zabs` which tests if the time is positive or not, it iterates over all drone and trip pairs, adding: `opt.add(Implies(And(Or(k1 != k2, t1 != t2), delivery_time[(k1,t1,i)] > 0, delivery_time[(k2,t2,i)] > 0), zabs(delivery_time[(k1,t1,i)] - delivery_time[ >= w))`.

```
def zabs(x):
    return If(x >= 0, x, -x)

for i in delivery_nodes:
    for k1 in drone_keys:
        for k2 in drone_keys:
            for t1 in trips:
                for t2 in trips:
                    # If either drone or time index differs and both have delivery_time>0 at i, ensure |diff
     |>=w
                    cond = And(Or(k1 != k2, t1 != t2),
                               delivery_time[(k1, t1, i)] > 0,
                               delivery_time[(k2, t2, i)] > 0)
                    opt.add(Implies(cond, zabs(delivery_time[(k1, t1, i)] - delivery_time[(k2, t2, i)]) >= w
     ))
```

- **Test 2:** Same logic, but uses: `dt1 = delivery_time[k1][t1][i]` `dt2 = delivery_time[k2][t2][i]` `opt.add(Implies(And(dt1 > 0, dt2 > 0), Or(dt1 - dt2 >= w, dt2 - dt1 >= w)))` and avoids using absolutes to, reducing solver complexities.

```
for i in delivery_nodes:
    for k1 in drone_ids:
        for t1 in trips:
            for k2 in drone_ids:
                for t2 in trips:
                    if not (k1 == k2 and t1 == t2):
                        # Replace Abs(dt1 - dt2) >= w with equivalent Or to avoid potential solver issues
                        dt1 = delivery_time[k1][t1][i]
                        dt2 = delivery_time[k2][t2][i]
                        opt.add(Implies(And(dt1 > 0, dt2 > 0),
                                        Or(dt1 - dt2 >= w, dt2 - dt1 >= w)))
```

- **Test 3:** Same as Attempt 1

Node-to-Node Arrival (Constraint 6)

- **Test 1**:

```
for k in drone_keys:
    for t in trips:
        for j in neighbors.get(source, []):
            opt.add(Implies(X_delivery[(k, t, source, j)],
                            arrival_time[(k, t, j)] == start_time[(k, t)] + get_edge_time(source, j)))
```

- **Test 2**: Same as Test 1

- **Test 3**: Same as Test 1

Node-to-Node Arrival (Constraint 7)

- **Test 1**: Implements arrival time propagation only for delivery nodes, using `opt.add(Implies(X_delivery[(k, t, i, j)], arrival_time[(k, t, j)] == arrival_time[(k, t, i)] + get_edge_time(i, j) + IntVal(w)))`. However this places the service time `w` onto non delivery nodes which will cause issues with timing.

```
for k in drone_keys:
    for t in trips:
        for i in delivery_nodes:
            for j in nodes:
                if (i, j) in edges and i != j:
                    opt.add(Implies(X_delivery[(k, t, i, j)],
                                    arrival_time[(k, t, j)] == arrival_time[(k, t, i)] + get_edge_time(i, j)
      + IntVal(w)))

```

- **Test 2**: Handles both delivery and non-delivery nodes, adding `w` for delivery nodes and excluding it for non-delivery nodes, using `opt.add(Implies(X_delivery[k][t][i][j], arrival_time[k][t][j] == arrival_time[k][t][i] + edges[(i, j)]['time'] + w))` or without `w`.

```
for k in drone_ids:
    for t in trips:
        for i in nodes:
            for j in nodes:
                if (i, j) in edges and i != j:
                    if i in delivery_nodes:
                        opt.add(Implies(X_delivery[k][t][i][j],
                                        arrival_time[k][t][j] == arrival_time[k][t][i] + edges[(i, j)]['time
      '] + w))
                    else:
                        opt.add(Implies(X_delivery[k][t][i][j],
                                        arrival_time[k][t][j] == arrival_time[k][t][i] + edges[(i, j)]['time
      ']))

```

- **Test 3**: Similarly handles both delivery nodes and non-delivery nodes, using `get_X` for efficient edge access, adding `w` for delivery nodes and excluding it for non-delivery nodes. The `get_X` function minimizes variable allocation, enhancing efficiency.

```
for k in drone_keys:
    for t in trips:
        for i in nodes:
            outs = [(j, get_X(k, t, i, j)) for j in nodes if (i, j) in E and i != j]
            for (j, x) in outs:
                if i in delivery_nodes:
                    opt.add(Implies(as_bool(x),
                                    arrival_time[(k, t, j)] == arrival_time[(k, t, i)] + edge_time[(i, j)] +
      w))
                else:
                    opt.add(Implies(as_bool(x),
                                    arrival_time[(k, t, j)] == arrival_time[(k, t, i)] + edge_time[(i, j)]))

```

Flow Conservation (8)

- **Test 1:** Implemented using `sum_bools` function to count outgoing `outdeg = sum_bools([X_delivery[(k,t,i,j)] for j in nodes])`, and incoming edges `indeg = sum_bools([X_delivery[(k,t,j,i)] for j in nodes])`, adding `opt.add(outdeg == indeg)`. This iterates over all nodes, generating constraints for all possible edges, including non-existent ones (constrained elsewhere to False), increasing computational load due to a dense constraint set.

```
def sum_bools(bs):
    return Sum([If(b, 1, 0) for b in bs]) if bs else IntVal(0)
    # 8. Flow conservation per node in a trip: outdeg == indeg
for k in drone_keys:
    for t in trips:
        for i in nodes:
            outdeg = sum_bools([X_delivery[(k, t, i, j)] for j in nodes])
            indeg = sum_bools([X_delivery[(k, t, j, i)] for j in nodes])
            opt.add(outdeg == indeg)
```

- **Test 2:** Implements similar to Test 1 but uses helper functions `sum_out(k,t,i)` and `sum_in(k,t,i)` which summarizes valid edges `if (i,j) in edges` reducing application of constraints.

```
def sum_out(k, t, i):
    return Sum([If(X_delivery[k][t][i][j], 1, 0) for j in nodes if (i, j) in edges and i != j])

def sum_in(k, t, i):
    return Sum([If(X_delivery[k][t][j][i], 1, 0) for j in nodes if (j, i) in edges and i != j])

# Flow conservation per node within trip
for k in drone_ids:
    for t in trips:
        for i in nodes:
            opt.add(sum_out(k, t, i) == sum_in(k, t, i))
```

- **Test 3:** Uses a custom `get_X(k,t,i,j)` function to access edge variables, returning `BoolValFalse` for non-existent edges. Outgoing edges are summarized with `sum_out = Sum([If(as_bool(get_X(k,t,i,j)), 1, 0) for j in nodes])` and incoming edges with `sum_in = Sum([If(as_bool(get_X(k,t,j,i)), 1, 0) for j in nodes])`. The constraint then uses both variables and inserted: `opt.add(sum_out == sum_in)`.

```
def get_X(k, t, i, j):
    if (i, j) in E:
        key = (k, t, i, j)
        if key not in X_delivery:
            X_delivery[key] = Bool(f"X_{k}_{t}_{i}_{j}")
        return X_delivery[key]
    # For non-edge, return constant False
    return BoolValFalse

for k in drone_keys:
    for t in trips:
        for i in nodes:
            # Self loop forbidden
            if (i, i) in E:
                opt.add(get_X(k, t, i, i) == False)
            # Flow conservation: sum out == sum in
            sum_out = Sum([If(as_bool(get_X(k, t, i, j)), 1, 0) for j in nodes])
            sum_in = Sum([If(as_bool(get_X(k, t, j, i)), 1, 0) for j in nodes])
            opt.add(sum_out == sum_in)
```

Max Arrival Time(9)

- **Test 1**:

```
# 9. If node i has any outgoing edge used, arrival_time <= t_max
for k in drone_keys:
    for t in trips:
        for i in nodes:
            outdeg = sum_bools([X_delivery[(k, t, i, j)] for j in nodes])
            opt.add(Implies(outdeg > 0, arrival_time[(k, t, i)] <= t_max))
```

- **Test 2**: Same as Test 1
- **Test 3**: Same as Test 1

No Self-Loops (10)

- **Test 1**: Implemented with edges that do not edges exist or a node to itself.

```
for k in K:
    for t in T:
        for i in I:
            for j in J:
                if (i, j) not in edges or i == j:
                    opt.add(X_delivery[(k, t, i, j)] == False)
```

- **Test 2**: Same logic as Test 1

- **Test 3**: Same as Test 1

Resource Delivery Balance (11)

- **Test 1**:

```
for i in delivery_nodes:
    req_i = int(resources_required.get(i, 0))
    delivered_sum = Sum([resources_delivered[(k, t, i)] for k in drone_keys for t in trips]) if req_i > 0
     else IntVal(0)
    opt.add(Implies(BoolVal(True), delivered_sum == IntVal(req_i)))
```

- **Test 2**:

```
for i in delivery_nodes:
    opt.add(Sum([resources_delivered[k][t][i] for k in drone_ids for t in trips]) == int(resources_required.
     get(i, 0)))
```

- **Test 3**:

```
for k in drone_ids:
    cap_k = int(drones[k].get('capacity', 0))
    for t in trips_range:
        hard.add(zsum_ints([resources_delivered[(k, t, i)] for i in delivery_nodes]) <= cap_k)
```

Resource Capacity Limit (12)

- **Test 1**:

```
for k in K:
    cap_k = int(drones[k].get('capacity', 0))
    for t in T:
        # sum delivered in this trip <= capacity
        sum_trip_deliv = Sum([resources_delivered.setdefault((k, t, i), Int(f"resources_delivered[{k},{t},{i
     }]")) for i in D])
        opt.add(sum_trip_deliv <= cap_k)
```

- **Test 2**:

```
 for k in drone_names:
    cap = int(drones_info[k].get('capacity', 0))
    for t in range(max_trips):
        o.add(Sum([resources_delivered.get((k, t, i), Int(f"_zero2_{k}_{t}_{i}")) for i in delivery_nodes])
     <= cap)
```

- **Test 3**:

```
for k in K:
    for t in T:
        sum_trip_deliv = Sum([resources_delivered[(k, t, i)] for i in delivery_nodes]) if delivery_nodes
     else IntVal(0)
        track(f"cap_trip[{k},{t}]", sum_trip_deliv <= IntVal(capacity[k]))
```

Source Load Balance (13), Delivery Node Load (14),Non-Deliver Node Load (15) and Source Resource Limit (16)

- **Test 1**:

```
# 13. load at source equals total delivered by drone
for k in drone_keys:
    opt.add(load[(k, source)] == Sum([resources_delivered[(k, t, i)] for t in trips for i in delivery_nodes
     ]))
# 14. load at delivery node i equals negative of sum delivered by k to i
for k in drone_keys:
    for i in delivery_nodes:
        opt.add(load[(k, i)] == -Sum([resources_delivered[(k, t, i)] for t in trips]))
# 15. load zero at transit (non-delivery) nodes except source
```

```
for k in drone_keys:
    for i in nodes:
        if i != source and i not in delivery_nodes:
            opt.add(load[(k, i)] == 0)
# 16. Resource availability at source
resources_available_source = int(resources_at.get(source, 0))
opt.add(Sum([load[(k, source)] for k in drone_keys]) <= resources_available_source)
```

- **Test 2**: Same as Test 1

- **Test 3**: Same as Test 1

Non-negative Delivery Amount (17)

- **Test 1**: Same as Test 1

```
for k in K:
    for t in T:
        for i in I:
            # No delivery to non-delivery nodes
            if i not in D:
                opt.add(resources_delivered[(k, t, i)] == 0)
            else:
                # 0 <= delivered <= min(capacity, remaining required upper bound)
                req_i = int(resources_required.get(i, 0))
                opt.add(resources_delivered[(k, t, i)] >= 0)
```

- **Test 2**: Same as Test 1

- **Test 3**: Same as Test 1

Trip Activation (18) and No Delivery Implies Inactive Trip(19)

- **Test 1**:

```
 # 18. If total demand > 0, at least one active trip exists
total_demand = sum([int(resources_required.get(i, 0)) for i in delivery_nodes])
if total_demand > 0:
    opt.add(Or([trip_active[(k, t)] for k in drone_keys for t in trips]))

# 19. If no resources delivered in a trip, then trip not active and no edges used
for k in drone_keys:
    for t in trips:
        sum_delivered_trip = Sum([resources_delivered[(k, t, i)] for i in delivery_nodes])
        sum_edges_used = sum_bools([X_delivery[(k, t, i, j)] for i in nodes for j in nodes])
        opt.add(Implies(sum_delivered_trip == 0, And(Not(trip_active[(k, t)]), sum_edges_used == 0)))

```

- **Test 2**: Same as Test 1

- **Test 3**:

```
for k in drone_keys:
    for t in trips:
        sum_del_vars = [resources_delivered[(k, t, i)] for i in delivery_nodes]
        if sum_del_vars:
            has_delivery = Sum(sum_del_vars) > 0
            # If trip has any delivery => trip_active, and if no delivery => not active and no edges
            opt.add(Implies(has_delivery, trip_active[(k, t)]))
            opt.add(Implies(Not(has_delivery), And(Not(trip_active[(k, t)]), Sum([If(as_bool(get_X(k, t, i,
     j)), 1, 0) for i in nodes for j in nodes]) == 0)))
        else:
            # No delivery nodes -> trip must be inactive and no edges
            opt.add(And(Not(trip_active[(k, t)]), Sum([If(as_bool(get_X(k, t, i, j)), 1, 0) for i in nodes
     for j in nodes]) == 0))

```

Time Trip Gap (20)

- **Test 1**:

```
for k in drone_keys:
    for t in trips:
        if t < max_trips - 1:
            opt.add(start_time[(k, t + 1)] >= start_time[(k, t)] + t_s)
```

- **Test 2**: Same as Test 1
- **Test 3**: Same as Test 1

Delivery Time Equals Arrival (21) and No Delivery, Zero Time (22):

- **Test 1**:

```
# 21. If delivered at node i in trip, delivery_time == arrival_time at i
for k in drone_keys:
    for t in trips:
        for i in delivery_nodes:
            opt.add(Implies(resources_delivered[(k, t, i)] > 0, delivery_time[(k, t, i)] == arrival_time[(k,
      t, i)]))
# 22. If no delivery at i in trip, delivery_time == 0
for k in drone_keys:
    for t in trips:
        for i in delivery_nodes:
            opt.add(Implies(resources_delivered[(k, t, i)] == 0, delivery_time[(k, t, i)] == 0))

```

- **Test 2**: Same logic as Test 1 just under one if statement

```
for k in drone_ids:
    for t in trips:
        for i in nodes:
            if i in delivery_nodes:
                pt.add(Implies(resources_delivered[k][t][i] > 0, delivery_time[k][t][i] == arrival_time[k][t
     ][i]))            opt.add(Implies(resources_delivered[k][t][i] == 0, delivery_time[k][t][i] == 0))
            else:
                # For non-delivery nodes, delivery_time must be 0
                opt.add(delivery_time[k][t][i] == 0)

```

- **Test 3**: Same Test 2

Visited Node Equals Traversed Edges (23)

- **Test 1:** Implements as `cnt_visited = sum_bools([visited_nodes[(k,t,i)] for i in nodes])`, `cnt_edges = sum_bools([X_delivery[(k,t,i,j)] for i in nodes for j in nodes])`, adding `opt.add(Implies(trip_active[(k,t)], cnt_visited == cnt_edges))`.

```
for k in drone_keys:
    for t in trips:
        cnt_visited = sum_bools([visited_nodes[(k, t, i)] for i in nodes])
        cnt_edges = sum_bools([X_delivery[(k, t, i, j)] for i in nodes for j in nodes])
        opt.add(Implies(trip_active[(k, t)], cnt_visited == cnt_edges))
```

- **Test 2:** Same logic, but uses `Sum([If(visited_nodes[k][t][i], 1, 0) for i in nodes])` and `Sum([If(X_delivery[k][t][i][j], 1, 0) for i in nodes for j in nodes if (i,j) in edges])`, filtering valid edges.

```
def ZOr(lst):
    if not lst:
        return BoolVal(False)
    return Or(*lst)
for k in drone_ids:
    for t in trips:
        for i in nodes:
```

```
            incident_used = ZOr(
                [X_delivery[k][t][i][j] for j in nodes if (i, j) in edges and i != j] +
                [X_delivery[k][t][j][i] for j in nodes if (j, i) in edges and i != j]
            )
            opt.add(visited_nodes[k][t][i] == incident_used)

# Active trip implies delivered > 0 and at least 1 delivery node visited
for k in drone_ids:
    for t in trips:
        num_visited = Sum([If(visited_nodes[k][t][i], 1, 0) for i in nodes])
        num_edges = Sum([If(X_delivery[k][t][i][j], 1, 0) for i in nodes for j in nodes if (i, j) in edges
     and i != j])
        opt.add(Implies(trip_active[k][t], num_visited == num_edges))
```

- **Test 3:** Identical logic, using `Sum([If(visited_nodes[(k,t,i)], 1, 0) for i in nodes])` and `Sum([If(as_bool(get_X(k,t,i,j)), 1, 0) for i in nodes for j in nodes])` with `get_X`.

```
for k in drone_keys:
    for t in trips:
        opt.add(Implies(trip_active[(k, t)],
            Sum([If(visited_nodes[(k, t, i)], 1, 0) for i in nodes]) ==
            Sum([If(as_bool(get_X(k, t, i, j)), 1, 0) for i in nodes for j in nodes])))
```

Successor for Visited Nodes (24)

- **Test 1**: Ensures visited non-source nodes have a successor matching an outgoing edge, using `opt.add(Implies(And(i != source, visited_nodes[(k, t, i)]), Or(choices)))` where `choices` pairs `X_delivery` and `successor`. It reduces the solver's search space by setting `visited_nodes[(k, t, i)] == False` for nodes with no outgoing edges using `outgoing.get(i, [])`, preventing invalid routes.

```
for k in drone_keys:
    for t in trips:
        for i in nodes:
            if i != source:
                if not outgoing.get(i, []):
                    opt.add(visited_nodes[(k, t, i)] == False)
                else:
                    choices = [And(X_delivery[(k, t, i, j)], successor[(k, t, i)] == j)
                               for j in nodes if (i, j) in edges]
                    opt.add(Implies(And(i != source, visited_nodes[(k, t, i)]), Or(choices)))

```

- **Test 2**: Uses `ZOr` to link visited nodes `visited_nodes` to outgoing edges, adding `opt.add(Implies(visited_nodes[k][t][i], ZOr([And(X_delivery[k][t][i][j], successor[k][t][i] == node_indices[j]) for j in nodes if (i, j) in edges])))` for non-source nodes, without handling nodes with no outgoing edges explicitly. The `ZOr` function handles empty edge lists by returning `BoolVal(False)`, but it does not explicitly constrain nodes with no outgoing edges, relying on other constraints (such as flow conversion) to prevent invalid visits.

```
def ZOr(lst):
    if not lst:
        return BoolVal(False)
    return Or(*lst)
for k in drone_ids:
    for t in trips:
        for i in nodes:
            if i != initial_node:
                opt.add(Implies(visited_nodes[k][t][i],
                                ZOr([And(X_delivery[k][t][i][j], successor[k][t][i] == node_indices[j])
                                     for j in nodes if (i, j) in edges and i != j])))

```

- **Test 3**:

```
for k in drone_ids:
            for t in trips_range:
                # For source when trip is active
                ors = []
                for j in nodes:
                    if edge_exists(source, j):
                        ors.append(And(X_delivery[(k, t, source, j)], successor[(k, t, source)] ==
    node_indices[j]))
                if ors:
                    hard.add(Implies(trip_active[(k, t)], Or(ors)))
                # For any visited i != source
                for i in nodes:
                    if i == source:
                        continue
                    ors_i = []
                    for j in nodes:
                        if edge_exists(i, j):
                            ors_i.append(And(X_delivery[(k, t, i, j)], successor[(k, t, i)] == node_indices[
    j]))
                    if ors_i:
                        hard.add(Implies(And(i != source, visited_nodes_var[(k, t, i)]), Or(ors_i)))


```

Source Successor (25)

- **Test 1:**

```
for k in drone_keys:
    for t in trips:
        choices = []
        for j in neighbors.get(source, []):
            choices.append(And(X_delivery[(k, t, source, j)], successor[(k, t, source)] == IntVal(
     node_indices[j])))
        if choices:
            opt.add(Implies(trip_active[(k, t)], Or(choices)))
```

- **Test 2:**

```
for k in drone_ids:
    for t in trips:
        # For source on active trip, there exists j with edge and successor[source] == idx(j)
                opt.add(Implies(trip_active[k][t],
        ZOr([And(X_delivery[k][t][initial_node][j], successor[k][t][initial_node] == node_indices[j]) for j
     in nodes if (initial_node, j) in edges and j != initial_node])))
```

- **Test 3:** Same as Test 1

Source Successor (26)

- **Test 1:**

```
for k in drone_keys:
    for t in trips:
        for i in nodes: opt.add(Implies(Not(visited_nodes[(k, t, i)]), successor[(k, t, i)] == -1))
```

- **Test 2:** Same as Test 1

- **Test 3:** Same as Test 1

Source Successor (27)

- **Test 1:**

```
for k in drone_keys:
    for t in trips:
        opt.add(Implies(trip_active[(k, t)], sum_bools([visited_nodes[(k, t, i)] for i in delivery_nodes])
     >= 1))
```

- **Test 2:** Same as Test 1

- **Test 3:** Same as Test 1

Active Trip Deliver Resource (28)

- **Test 1:**

```
for k in drone_keys:
    for t in trips:
        opt.add(Implies(trip_active[(k, t)], Sum([resources_delivered[(k, t, i)] for i in delivery_nodes]) >
      0))
```

- **Test 2:** Same as Test 1

- **Test 3:** Same as Test 1

Sequential Trips: (29)

- **Test 1:**

```
for k in drone_keys:
    for t in trips:
        if t < max_trips - 1:
            opt.add(Implies(trip_active[(k, t)], start_time[(k, t + 1)] >= arrival_time[(k, t, source)]))
```

- **Test 2:** Same as Test 1

- **Test 3:** Same as Test 1

Return to Source (30)

- **Test 1:** Implemented using `sum_bools([X_delivery[(k,t,j,source)] for j in nodes])`, ensuring one incoming edge to the source node.

```
for k in drone_keys:
    for t in trips:
        indeg_source = sum_bools([X_delivery[(k, t, j, source)] for j in nodes])
        opt.add(Implies(trip_active[(k, t)], indeg_source == 1))
```

- **Test 2**: Same as Test 1

- **Test 3**: Same as Test 1

Energy Usage Calculation (31)

- **Test 1**:

```
# Energy usage
for k in K:
    energy_k = int(drones[k].get('energy', 0))
    sum_energy = []
    for t in T:
        for i in I:
            for j in outgoing.get(i, []):
                sum_energy.append(If(X[(k, t, i, j)], dist_of(i, j), 0))
    opt.add(energy_used[k] == Sum(sum_energy) if sum_energy else energy_used[k] == 0)
    opt.add(energy_used[k] <= energy_k)

```

- **Test 2**: Same as Test 1

- **Test 3**: Same as Test 1

Energy Capacity Limit (32)

- **Test 1**:

```
for k in drone_keys:
    opt.add(energy_used[k] <= int(drones[k].get('energy', 0)))
```

- **Test 2**: Same as Test 1
- **Test 3**: Same as Test 1

Minimize Total Energy (33) and Minimize Active Trips (34)

- **Test 1**:

```
# Objectives
total_energy = Sum([energy_used[k] for k in drone_keys]) if drone_keys else IntVal(0)
total_trips_active = Sum([If(trip_active[(k, t)], 1, 0) for k in drone_keys for t in trips]) if drone_keys
     else IntVal(0)
opt.minimize(total_energy)
opt.minimize(total_trips_active)

```

- **Test 2**: Same as Test 1
- **Test 3**: Same as Test 1

Analysis Other than synthatical differences, most of the code implemented are the same

7.3 Results Comparison

7.3.1 Results from using Natural language constraints

Test 1

```
Execution Result:
Satisfiable
Solve time: 816.32 seconds
Best Solution:
Drone drone1:
  No delivery trips
Drone drone2:
  No delivery trips
Drone drone3:
Energy used: 38
Number of delivery trips: 3

Trip Summary:
Trip  Path                          Start  End    Delivery Times       EN Used Current EN  Resources Held Required Amount
-------------------------------------------------------------------------------------------------------------------------
1     c->a, a->c                    6      25     a@13                  14      36          3              a(0), b(2), d(2), e(1)
2     c->e, e->d, d->c              29     55     d@45, e@37            16      20          3              a(0), b(2), d(0), e(0)
3     c->b, b->c                    114    127    b@118                 8       12          2              a(0), b(0), d(0), e(0)

Overall summary:
  Total resources delivered: {'a': 3, 'b': 2, 'd': 2, 'e': 1}
  Total energy used: 38
  Total trips: 3
  Resources required: {'a': 3, 'b': 2, 'd': 2, 'e': 1}
```

Test 1 Solution from Natural Language Constraints

Test 2

```
Execution Result:
Satisfiable
Solve time: 24.82 seconds
Best Solution:
Drone drone1:
No delivery trips

Drone drone2:
Energy used: 8
Number of delivery trips: 1

Trip Summary:
Trip  Path                          Start  End    Delivery Times       EN Used Current EN  Resources Held Required Amount
-------------------------------------------------------------------------------------------------------------------------
1     b->c, c->b                    4      12     a@4, e@5               8       67          2              a(2), b(2), d(2), e(0)

Drone drone3:
Energy used: 16
Number of delivery trips: 2

Trip Summary:
Trip  Path                          Start  End    Delivery Times       EN Used Current EN  Resources Held Required Amount
-------------------------------------------------------------------------------------------------------------------------
1     b->c, c->b                    13     26     a@1019, b@17, d@1001   8       42          3              a(2), b(1), d(1), e(1)
2     b->c, c->b                    987    1000   a@989, b@991, d@1      8       34          3              a(1), b(0), d(0), e(1)

Overall summary:
  Total resources delivered: {'a': 3, 'b': 2, 'd': 2, 'e': 1}
  Total energy used: 24
  Total trips: 3
  Resources required: {'b': 2, 'a': 3, 'd': 2, 'e': 1}
```

Test 2 Solution from Natural Language Constraints

Test 3

```
Execution Result:
Satisfiable
Solve time: 784.83 seconds
Best Solution:
Drone drone1:
  No delivery trips
Drone drone2:
Energy used: 8
Number of delivery trips: 1

Trip Summary:
Trip  Path                          Start  End    Delivery Times       EN Used Current EN  Resources Held Required Amount
-------------------------------------------------------------------------------------------------------------------------
1     c->b, b->c                    987    1000   b@991                  8       67          2              a(3), b(0), d(2), e(1)

Drone drone3:
Energy used: 30
Number of delivery trips: 2

Trip Summary:
Trip  Path                          Start  End    Delivery Times       EN Used Current EN  Resources Held Required Amount
-------------------------------------------------------------------------------------------------------------------------
1     c->e, e->d, d->c              946    972    e@954, d@962           16      34          3              a(3), b(2), d(0), e(0)
2     c->b, b->a, a->c              976    1000   a@988                  14      20          3              a(0), b(2), d(0), e(0)

Overall summary:
  Total resources delivered: {'a': 3, 'b': 2, 'd': 2, 'e': 1}
  Total energy used: 38
  Total trips: 3
  Resources required: {'a': 3, 'b': 2, 'd': 2, 'e': 1}
```

Test 3 Solution from Natural Language Constraints

The experimental results of three tests using natural language to generate Z3 constraints for a drone delivery scheduling problem produced solutions that satisfied the resource delivery requirements. Despite the three tests stating their satisfiability, only Test 1 and 3 yielded feasible solutions, while Test 2 violated several constraints causing major issues in its results. All Tests outputs show that they satisfied the resource delivery requirements (a: 3, b: 2, d: 2, e: 1) using three trips, however they differ in drone usage, energy consumption, timing consistency and adherence to the constraints.

Drone1 is unused in all Tests, likely due to its limited capacity, insufficient to meet any node's demand alone. Despite the Active Trip Deliver Resource (28) being missing for Test 3, the drones are still delivering.

Test 1 and 3 as seen in 7.3.1 and 7.3.1 both have different usages in drones with the former solely relying on drone3 and latter having drone2 and drone3. Both adhering to capacity, energy, and timing constraints (t_max = 1000, t_s = 4, w = 5). Notably Test 3's start time begins much later (946–987) nearing the end of the maximum allotted time which is still valid as there is no constraint which urges the solver to solve it in the shortest amount of time.

In contrast, Test 2 as seen in 7.3.1, utilizes drone2 and drone3 with very noticeable and inconsistent timing and routing issues of its drones violating seven constraints:

- **Source Outgoing Route (4)**: In all trips the routes start with node 'b' when the source node is supposed to be node 'c'.
- **Source-to-Node arrival (6)**, drone2 starts at time 4 with path b− >c, c− >b but delivers to node 'a' at time 4 and node 'e' at time 5. These nodes are not in the path, and the path c− >a takes 7 time units, making the timing impossible.
- **Flow Conservation (8)**: The path b− >c, c− >b for all trips, with deliveries to 'a', 'e', and 'd', suggests unequal incoming and outgoing routes at these nodes, violating flow conservation.
- **Max Arrival Time (9)**: drone3 delivers to 'a' at time 1019 and 'd' at time 1001, exceeding the maximum allowed time t_max = 1000.
- **Delivery Time Equals Arrival (21)**: drone2 delivers to a@4 and e@5, but these nodes are not visited (no arrival times), so delivery times cannot equal arrival times as required. This is the same situation with drone3 that delivers to a@1019, b@17, d@1001 in its first trip and to a@989, b@991, d@1 in its second. Only node 'b' is visited.
- **No Delivery, Zero Time (22)**: Based on the path of the drones, nodes 'a', 'e', and 'd' are not visited therefore should not have delivery times.
- **Visited Node Equals Traversed Edges (23)**: Deliveries occur to 'a', 'e', and 'd', but the path b− >c, c− >b only includes nodes 'b' and 'c'. This constraint is missing during its implementation causing the infeasible solution.

While Test 2 has the major issues, Test 3 also has a minor problem.

- Delivery Time Separation(5): Both drone2 and drone3 return to the source node 'c' at time 1000. This would violate the constraint where in the generated constraint does not consider the source is treated as a delivery node, but this simultaneous return is a problem design oversight.

Based on the results, Test 2's faster solve time is likely the result incomplete constraint enforcement, allowing an invalid solution, while Tests 1 and 3's longer solve times indicate thorough evaluation.

Overall, The natural language approach, while effective in Tests 1 and 3, showcase that the constraints are susceptible to mistranslation by the LLM, as seen in Test 2.

7.3.2 Results from using Pseudocode

Test 1

```
Execution Result:
Satisfiable
Solve time: 9.86 seconds
Best Solution:
Drone drone1:
Energy used: 8
Number of delivery trips: 1

Trip Summary:
Trip  Path                          Start  End    Delivery Times       EN Used Current EN  Resources Held Required Amount
-------------------------------------------------------------------------------------------------------------------------
1                                   31    44    d@0                  8       92         1             a(3), b(2), d(1), e(1)

Drone drone2:
No delivery trips

Drone drone3:
No delivery trips

Overall summary:
  Total resources delivered: {'a': 0, 'b': 0, 'd': 1, 'e': 0}
  Total energy used: 8
  Total trips: 1
  Resources required: {'a': 3, 'b': 2, 'd': 2, 'e': 1}
```

Test 1 Solution from Pseudocode Constraints

Test 2

```
Execution Result:
Unsatisfiable
Reason: Solver crashed: exception: access violation reading 0x00000251FB6D56AC
Solve time: 4407.48 seconds
```

Test 2 Solution from Pseudocode Constraints

Test 3

```
Execution Result:
Satisfiable
Solve time: 11.23 seconds
Best Solution:
Drone drone1:
Energy used: 8
Number of delivery trips: 1

Trip Summary:
Trip  Path                          Start  End    Delivery Times       EN Used Current EN  Resources Held Required Amount
-------------------------------------------------------------------------------------------------------------------------
1     b->c, c->b                    895    908    e@982                  8       92          1             a(3), b(2), d(2), e(0)

Drone drone2:
No delivery trips

Drone drone3:
No delivery trips

Overall summary:
  Total resources delivered: {'a': 0, 'b': 0, 'd': 0, 'e': 1}
  Total energy used: 8
  Total trips: 1
  Resources required: {'b': 2, 'a': 3, 'd': 2, 'e': 1}
```

Test 3 Solution from Pseudocode Constraints

The experimental results from three tests using pseudocode to generate Z3 con-

straints reveal significant challenges in achieving feasible solutions with only 2 Test of the 3 in achieving a satisfiable solution which is still however incomplete. Despite the generated constraints being similar in its formulation as pseudocode's structured syntax to ensure similar constraint definitions allowing the pseudocode to directly map with Z3 constructs(example: ForAll, Implies), the outcome between the 3 Tests varied significantly. Test 1 produced a satisfiable but infeasible solution, using only drone1 for one trip to deliver only one resource to d. Not only is it not actually satisfiable , but also in violation of four constraints:

- **Resource Delivery Balance (11)**: Only to node 'd': 1 is delivered, failing to meet demands (a: 3, b: 2, d: 2, e: 1) which should make more drones to deliver the resources.
- **Delivery Time Equals Arrival (21)**: The delivery time to d@0 cannot equal the arrival time at node 'd' as the start time is after start time 31 plus travel time.
- **No Delivery, Zero Time (22)**: Non-zero delivery time d@0 is assigned, but no deliveries are made to node 'a', 'b', or 'e', which should have zero delivery times, which might be a potential error in other nodes' delivery times.

Other constraints such as flow conservation (9) and successor-related constraints (24–26) may also have been violated but due to limited path details this prevents proper confirmation.

Test 2 resulted in a solver crash likely due to the constraint set being deeply nested recursive relations such as Start Time Gap (3) and Delivery Time Separation (5) introducing computational overhead leading to the solver running out of memory.

Test 3 produced a satisfiable but similarly infeasible solution, using drone1 for one trip delivering one unit of resource e along path b-¿c, c-¿b. This also violates:Resource Delivery Balance (11):

The pseudocode approach ensured consistent constraint formulations across tests, as seen in **??** the nearly identical Z3 constraints. However, the precision of pseudocode revealed flaws in the problem formulation, such as incomplete resource delivery and timing inconsistencies, leading to infeasible or unsatisfiable results.

Based on the test, the natural language approach demonstrated greater success in producing solutions, with 2 out of 3 solutions being feasible while the Pseudocode approach was not able to produce it any proper ones. This showcases the LLM's ability in interpreting complex constraints when expressed in natural language and can translate them into workable formulations however it is also still susceptibility to mistranslations.

7.4 Limitations and Future Work

As mentioned in , there is oversight in one of the constraints leading to a possible error.

Limitations: Bounded domains limit scalability; LLM hallucinations mitigated by validation/retries. The experiment that generated Z3 constraints for the drone delivery scheduling problem using natural language and pseudocode revealed a number of challenges that affected the scalability and dependability of the solutions.1. Limitation

LLMs' mistranslation: Although the natural language approach worked well in Tests 1 and 3, it was vulnerable to mistranslation, as seen in Test 2 (7.3.1), where improper routing and timing resulted in the violation of seven criteria. As noted in 7.4, a significant flaw in the natural language approach is how the Delivery Time Separation limitation (5) is handled. Due to improper treatment of the source as a delivery node, simultaneous drone returns to the source node at time 1000 in Test 3 violated this constraint.

8 Conclusion

Based on the results of the experiment. The LLM is inconsistent with being able to translate the constraints from pseudocode but more consistent with being able to translate the constraints in natural language. Its possible that the written pseudocode is not written correctly thus also confusing the LLM at translating the constraints needed for the problem. The problem mainly comes from trying to encode the constraints and allow Z3 to solve the problem Possible improvements to the system could include implementing time-based constraints, such as dynamically adjusting edge costs or availability during certain periods of time like "At 15 to 35 units time, the cost for edges $b->a$ and $c->e$ are doubled and edge $e->d$ is unusable" to represent storms or typhoons. Priority deliveries with deadlines or for delivery time windows could also be implemented to be more realistic. To increase the robustness of the system, when the model is created, let the user add more information using a re-prompt loop. Then, run the model again with the new information, and if any mistakes occur, have the LLM repair the model again.

The experiment demonstrated the need for advanced LLM prompting techniques or hybrid validation to guarantee consistent satisfiability, even if the natural language approach showed promise. For improved scalability in UAV routing issues, future research should experiment with other solvers like CPLEX or OR-Tools.

References

[CEKT25] Diderot D. Cidjeu, Jean Louis Kedieng Ebongue Fendji, Vivent Corneille Kamla, and Igor Tchappi. Uav leveraging genai/llms, a brief survey. *Procedia Computer Science*, 265:382–389, 2025. 20th International Conference on Future Networks and Communications/ 22nd International Conference on Mobile Systems and Pervasive Computing/15th International Conference on Sustainable Energy Information Technology (FNC/MobiSPC/-SEIT 2025).

[dMB08] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient smt solver. In C. R. Ramakrishnan and Jakob Rehof, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg.

[GC25] Mingyang Geng and Anping Chen. Optimized scheduling for multi-drop vehicle–drone collaboration with delivery constraints using large language

models and genetic algorithms with symmetry principles. *Symmetry*, 17(6), 2025.

[HCL+25] Biao Han, Yitang Chen, Jinrong Li, Jie Li, and Jinshu Su. Swarmchain: Collaborative llm inference for uav swarm control. *IEEE Internet of Things Magazine*, pages 1–9, 2025.

[HX25] Xiangyu Hu and Zhenyu Xu. Large language and vision-language models for robot: safety challenges, mitigation strategies and future directions. *Industrial Robot: the international journal of robotics research and application*, 07 2025.

[JFHS24] Shumaila Javaid, Hamza Fahim, Bin He, and Nasir Saeed. Large language models for uavs: Current state and pathways to the future. *IEEE Open Journal of Vehicular Technology*, 5:1166–1192, 2024.

[JPDB25] Matheus B. Jenevain, Milena F. Pinto, Mario A. R. Dantas, and Laís R. Berno. Using uavs and retrieval augmented generation for situational awareness in rescue operations. In Leonard Barolli, editor, *Advanced Information Networking and Applications*, pages 143–153, Cham, 2025. Springer Nature Switzerland.

[LMSK24] Jorge Lovaco, Raghu Munjulury, Ingo Staack, and Petter Krus. Large language model-driven simulations for system of systems analysis in fire-fighting aircraft conceptual design. 09 2024.

[MKBM25] Swarnamouli Majumdar, Sonny E. Kirkley, and Biswadip Basu Mallik. Llm-guided hybrid architecture for autonomous fire response: Dialog-driven planning in space and disaster missions. In *2025 IEEE World AI IoT Congress (AIIoT)*, pages 1049–1054, 2025.

[RGDB+21] G. Radzki, P. Golinska-Dawson, G. Bocewicz, et al. Modelling robust delivery scenarios for a fleet of unmanned aerial vehicles in disaster relief missions. *Journal of Intelligent & Robotic Systems*, 103(63), 2021.

[RWRL24] Alok Rawat, Emlyn Witt, Mohamad Roumyeh, and Irene Lill. Advanced digital technologies in the post-disaster reconstruction process—a review leveraging small language models. *Buildings*, 14(11), 2024.

[Sez25] Anıl Sezgin. Scenario-driven evaluation of autonomous agents: Integrating large language model for uav mission reliability. *Drones*, 9:213, 03 2025.

[SLA+25] Kumpol Saengtabtim, Natt Leelawat, Rujapa Aumnoysombat, Mahsa Adelifar, Natcha Saengwongwattana, Gritaya Suktavornprasit, Anawat Suppasri, Fumihiko Imamura, and Jing Tang. Harnessing generative ai for enhanced disaster management: a systematic review. *Big Earth Data*, 0(0):1–25, 2025.

[Wie14] Roel J. Wieringa. *What Is Design Science?*, pages 3–11. Springer Berlin Heidelberg, Berlin, Heidelberg, 2014.

[ZLZ+24] Wen Zhao, Liqiao Li, Hanwen Zhan, Yingqi Wang, and Yiqi Fu. Applying large language model to a control system for multi-robot task assignment. *Drones*, 8(12), 2024.